

دراسة وتنفيذ محطة مراقبة بيئية
ذكية
مبنية على STM32 مع المحاكاة
الافتراضية

□□□□□□□ □□□□□ □□□□

10 ماي و 2026

المخلص

تُمثل ثورة تكنولوجيا المعلومات الجديدة التي تقودها إنترنت الأشياء والتقنيات المدمجة إعادة تشكيل جذريّ لأنماط تشغيل المجتمعات، وأصبحت المراقبة البيئية الذكية مكوّنًا أساسيًا في بناء المدن الذكية. ومع ذلك، يواجه المطوّرون بشكل عام خللًا عمليًا في التشغيل والتطوير في الأنظمة المدمجة تحديات عديدة من بينها التكلفة العالية للعتاد، وتعقيد عملية التصحيح، والحاجة إلى فهم معمّق للحساسات، وصعوبة تصور البيئات. يتطلب نمط التطوير المدمج التقليدي من المتعلمين شراء كميات كبيرة من المعدات المادية وإجراء اختبارات متكررة على اللوحات الدوائرية الفعلية، مما يزيد من تكلفة التعلم ويحد من مرونة وقابلية التوسع في سيناريوهات التدريب. لمعالجة المشكلات المذكورة أعلاه، تقدّم هذه الورقة البحثية نظام محاكاة افتراضية لمحطة مراقبة بيئية ذكية مبنية على STM32. تشمل المساهمات الأساسية لهذا البحث ثلاثة جوانب: أولاً، تم تصميم وتنفيذ منصة عتاد افتراضية عالية الدقة لـ STM32F103C8T6 تحاكي بالكامل نواة MCU ونظام الذاكرة ووحدات GPIO و ADC و I2C و UART والمؤقتات وغيرها من الملحقات الأساسية، مما يوفر بيئة برمجية يمكن تشغيل البرامج المدمجة فيها دون الحاجة إلى عتاد حقيقي. ثانياً، تم تطوير برامج تشغيل افتراضية لسلسلة أنواع شرائعة من الحساسات البيئية بناءً على المنصة الافتراضية، تشمل حساس DHT11 لدرجة الحرارة والرطوبة، وحساس MQ-135 لجودة الهواء، وحساس BH1750 للإضاءة، وحساس BMP280 للضغط الجوي، وحساس الضوضاء، مع إنشاء إطار موحد لنمذجة الحساسات ونموذج الضوضاء لضمان واقعية البيانات المحاكاة وموثوقيتها. ثالثاً، تم بناء نظام تصور ويب مبني على إطار عمل Flask، Python يحقق جمع بيانات الحساسات في الوقت الفعلي وتخزينها وإنذارها وعرضها في شكل رسوم بيانية، ويتفاعل مع الواجهة الأمامية عبر واجهة برمجة تطبيقات RESTful، مع دعم التحكم عن بُعد في المعلمات البيئية.

أظهرت نتائج التجارب أن كفاءة تنفيذ التشغيلات للمنصة الافتراضية STM32 تتمتع بدرجة عالية من التوافق مع العتاد الحقيقي، ووصلت دقة محاكاة الحساسات إلى القيمي المعلنة في أوراق البيانات أو اقتربت منها، وكان وقت استجابة واجهة برمجة التطبيقات لنظام الويب أقل من 100 مللي ثانية، مما يفي بمتطلبات المراقبة في الوقت الفعلي. يوفر هذا البحث للتدريس في الأنظمة المدمجة منصة تجريبية منخفضة الحواجز ومرنة للغاية، كما يوفر بيئة تطوير برمجية فعّالة للتحقق من النماذج الأولية لتطبيقات إنترنت الأشياء، مما أنتج قيمة إيجابية على المستويين النظري والهندسي.

الكلمات المفتاحية: STM32 المحاكاة الافتراضية؛ المراقبة البيئية؛ محاكاة الحساسات؛ إنترنت الأشياء؛ تدريّس الأنظمة المدمجة

المحتويات

1	المُلخَص
5	1 المقدمة
5	1.1 خلفية البحث وأهميته
6	1.2 الحالة الحالية للبحث في الداخل والخارج
6	1.2.1 البحث في منصات المحاكاة المدمجة
7	1.2.2 البحث في أنظمة المراقبة البيئية
8	1.3 الأعمال الرئيسية للورقة البحثية
8	1.4 هيكّل الورقة البحثية
9	2 التصميم العام للنظام
9	2.1 تحليل متطلبات الوظائف
9	2.1.1 المتطلبات الوظيفية
10	2.1.2 المتطلبات غير الوظيفية
11	2.2 التصميم المعماري العام
12	2.3 اختيار التقنية
13	3 تصميم منصة عتاد STM32 الافتراضية
14	3.1 تصميم نواة MCU الافتراضية
14	3.1.1 نظرة عامة على معلمات STM32F103C8T6
14	3.1.2 محاكاة مؤقت SysTick
15	3.2 تصميم نظام الذاكرة
15	3.2.1 نموذج تعيين الذاكرة
16	3.2.2 تنفيذ نمط Little-Endian
16	3.2.3 تعيين المسجلات
17	3.3 محاكاة الملحقات
17	3.3.1 GPIO للإدخال/الإخراج العام
17	3.3.2 ADC لمحول التماثل إلى الرقمي
18	3.3.3 وحدة تحكم I2C bus
19	3.3.4 UART للاتصال التسلسلي
20	3.3.5 TIM للمؤقت العام
22	4 برامج تشغيل الحساسات وجمع البيانات
22	4.1 مبادئ المحاكاة الافتراضية للحساسات
22	4.1.1 إطار النمذجة العام
24	4.1.2 واجهة برنامج تشغيل الحساس
24	4.2 حساس DHT11 لدرجة الحرارة والرطوبة

24	نظرة عامة على الحساس	4.2.1
24	محاكاة بروتوكول bus أحادي	4.2.2
26	حساس MQ-135 لجودة الهواء	4.3
26	مبدأ الحساس	4.3.1
26	الاستدلال العكسي من الجهد إلى التركيز	4.3.2
27	تنفيذ برنامج التشغيل الافتراضي	4.3.3
27	حساس BH1750 للإضاءة	4.4
27	نظرة عامة على الحساس	4.4.1
28	صيغة حساب الإضاءة	4.4.2
28	حساس BMP280 للضغط الجوي	4.5
28	نظرة عامة على الحساس	4.5.1
29	التحقق من معرف الشريحة	4.5.2
29	ADC ذو 24 بت ومعاملات المعايرة	4.5.3
30	حساس الضوضاء	4.6
30	مبدأ الحساس	4.6.1
30	التعويض الخطي للجهد	4.6.2
31	تصميم دورة جمع البيانات	4.7
33	5 تصميم النظام البرمجي وتنفيذه	
33	وحدة تخزين البيانات	5.1
33	تصميم هيكل جداول قاعدة البيانات	5.1.1
35	واجهة sqlite3 في Python	5.1.2
35	عمليات CRUD	5.1.3
35	استعلامات إحصائية	5.1.4
36	تصميم نظام التنبيه	5.2
36	تكوين حدود التنبيه	5.2.1
37	منطق فحص التنبيه	5.2.2
39	تخزين سجلات التنبيه بشكل دائم	5.2.3
39	نظام تصور الويب	5.3
39	تصميم مسارات Flask	5.3.1
40	تنسيق استجابة RESTful JSON	5.3.2
41	تكوين مخطط الخط المزدوج في Chart.js	5.3.3
42	آلية الاستطلاع كل ثانية	5.3.4
43	تصميم لوحة التحكم	5.4
43	تصميم آلية IPC	5.4.1
44	تحليل أمر set	5.4.2
45	قيود نطاق المعاملات	5.4.3

45

6 اختبار النظام والتحقق

46	6.1	بيئة الاختبار
46	6.2	الاختبار الوظيفي
48	6.3	اختبار الأداء
48	6.3.1	زمن استجابة API
48	6.3.2	استقرار دورة الجمع
49	6.3.3	استهلاك الذاكرة
49	6.4	التحقق من دقة الحساس
49	6.4.1	تحليل مقارنة الدقة
50	6.4.2	اختبار الاستقرار على المدى الطويل
50	6.5	التحقق من وظيفة التنبؤ
51	6.5.1	سيناريوهات تنبؤ الحرارة العالية
51	6.5.2	سيناريوهات تنبؤ الضغط المنخفض
52	6.5.3	سيناريوهات تنبؤات متزامنة متعددة
52	6.6	عرض واجهة الويب

53

7 الخلاصة والآفاق

53	7.1	ملخص أعمال الورقة البحثية
54	7.2	آفاق العمل المستقبلي

56

المراجع

1 المقدمة

1.1 خلفية البحث وأهميته

في الوقت الحاضر، تُعיד ثورة تكنولوجيا المعلومات الجديدة التي يقودها إنترنت الأشياء (IoT) Things, of (Internet) تشكّل أنماط تشغيل المعلومات. إن تحقيق تطبيقات المدن الذكية والأتمتة الصناعية والزراعة الدقيقة والمنزل الذكية يعتمد جميعه على المراقبة المستمرة للمعلومات الفيزيائية للبيئة. إن مؤشرات درجة الحرارة والرطوبة وجودة الهواء وشدة الإضاءة والضغط الجوي ومستوى الضوضاء لا تتعلق فقط بصحة وراحة الأفراد، بل تؤثر أيضاً على كفاءة الإنتاج في البيئات الصناعية. لذلك، فإن تطوير حلول مراقبة بيئية مستقرة ومنخفضة التكلفة ومرنة في النشر له قيمة ملحوظة في بعدي الخدمة الاجتماعية والتشغيل الاقتصادي.

تشكّل أنظمة الحوسبة المدمجة الأساس التقني لعقد الحافة في إنترنت الأشياء، وتتحمل مسؤولية الحصول على بيانات الاستشعار ومعالجتها مسبقاً ونقلها عبر الشبكة. تتمتع عائلة منتجات STM32 التي طرحتها شركة STMicroelectronics، وخاصة طراز STM32F103C8T6 المجزأ بنواة Cortex-M3 بتوازن جيد بين الأداء الحسابي واستهلاك الطاقة، وتنوع موارد الملحقات على الشريحة، ونضج سلسلة الأدوات المصاحبة، مما أدى إلى ارتفاع معدل التبني في التعليم الجامعي والممارسة الهندسية. وقد تراكمت خبرات تطبيقات بيئية غنية لهذا الشريحة في مجالات فرعية تشمل المراقبة البيئية والتحكم الصناعي وإلكترونيات المستهلك. ومع ذلك، سواء في البيئات التعليمية أو التعلم الذاتي، يواجه دخول الأنظمة المدمجة وتقديمها بعض العوائق الواقعية:

(1) **التكلفة العالية للشراء.** تتطلب مجموعة مراقبة بيئية كاملة شراء لوحة تطوير STM32 ووحدات حساسات متنوعة وأدوات تصحيح ومصدر طاقة وكوابل توصيل، حيث تتراوح تكلفة المجموعة الواحدة عادةً بين مئات وآلاف الليرات. بالنسبة للمختبرات الجامعية، يتطلب توفير عشرات المجموعات التجريبية الكاملة استثماراً مالياً كبيراً؛ وبالنسبة للمتعلمين الأفراد، تشكل التكلفة المرتفعة للبدء حاجزاً معيئاً.

(2) **تعقيد عملية التصحيح.** يعتمد تصحيح الأنظمة المدمجة على حالة الاتصال الفعلية للدوائر المادية، حيث يمكن لأي كبل مفكوك أو مقاوم ملحوم بشكل خاطئ أو خطأ في تهئية pin أن يتسبب في فشل النظام في العمل بشكل صحيح. عند مواجهة أعطال متتالية، يجد المبتدئون صعوبة في تحديد جذر المشكلة بسرعة، مما يستهلك وقتاً طويلاً في استكشاف أخطاء العتاد بدلاً من التركيز على تعلم الخوارزميات البرمجية ومنطق النظام.

(3) **الحاجة إلى فهم عميق للحساسات.** تختلف مبادئ عمل الحساسات المختلفة وبروتوكولات الاتصال وطرق معالجة البيانات. على سبيل المثال، يستخدم DHT11 بروتوكول bus أحادي لنقل 40 بت من البيانات، ويتطلب MQ-135 جمع الجهد عبر ADC والاعتماد على نماذج رياضية معقدة للاستدلال العكسي على

تركيز الغاز، بينما يتضمن BMP280 اتصال I2C وقراءة ADC ذات 24 بت وخوارزمية تعويض الحرارة. يتطلب الفهم العميق لآليات عمل هذه الحساسات من المتعلمين امتلاك معرفة واسعة في الدوائر التناظرية والرقمية وبروتوكولات الاتصال ومعالجة البيانات.

(4) **صعوبة تصورات البيانات.** في بيئات التطوير المدمجة التقليلية، يتم عادةً إخراج بيانات الحساسات عبر المنفذ التسلسلي بتنسيق نصي، ويحتاج المطورون إلى عرض القيم الخام في مساعد المنفذ التسلسلي، مما يفتقر إلى وسائل تصورات بيانية بديهية. حتى مع مساعدة برامج الكمبيوتر والعلي، غالباً ما تتطلب وظائف إضافية، وتكون القدرات محدودة، ومن الصعب تحقيق وظائف متقدمة مثل الوصول عن بُعد عبر الويب واستعلام البيانات التاريخية والتنبؤ به الذكي.

بناءً على التحليل أعلاه، فإن تطوير نظام محاكاة افتراضية لمحطة مراقبة بيئية ذكية مبنية على STM32، مما يُمكّن المتعلمين من إكمال التدريب الكامل لتطوير الأنظمة المدمجة دون شراء عتاد مادي، له قيمة مزدوجة: من ناحية، يوفر حاملاً تجريبياً منخفض التكلفة لنظام المقرر الدراسي؛ ومن ناحية أخرى، يوفر بيئة تكرار سريع للتحقق المبكر من المفاهيم.

1.2 الحالة الحالية للبحث في الداخل والخارج

1.2.1 البحث في منصات المحاكاة المدمجة

بدأ البحث في مجال المحاكاة الافتراضية للمعالجات والأنظمة المدمجة في وقت مبكر، وقد بذلت الأوساط الأكاديمية والصناعية جهوداً كبيرة. يُعد Quick QEMU (Emulator) محاكي عام مفتوح المصدر للآلات، وهو متوافق مع العديد من بنى مجموعات التشغيل بما في ذلك سلسلة Cortex-M. ARM. تدعم آليته الأساسية على الترجمة الثنائية الديناميكية، حيث يمكنها تحويل تدفق تعليمات المنصة المستهدفة إلى تدفق تعليمات المضيف لتنفيذها بكفاءة، وبالتالي يتم استخدامها بشكل متكرر في مراحل نقل أنظمة التشغيل والتحقق من البرامج المدمجة. ومع ذلك، يركز محاكاة QEMU على سلوك نواة CPU ونظام التشغيل، ولا يزال دعم نمذجة الملحقات الدقيقة للمتحكمات الدقيقة (GPIO، ADC، TIM، إلخ) غير كامل، كما أن عملية التكوين ومنحني التعلم غير وديين بما فيه الكفاية للسيناريوهات التشغيلية والتطوير السريع للنماذج الأولية.

تم تطوير Proteus بواسطة شركة Electronics Labcenter البريطانية، وهو أداة أتمتة التصميم الإلكتروني (EDA) مع إمكانيات مدمجة لمحاكاة الدوائر وتخطيط PCB والتصحيح المشترك للمتحكمات الدقيقة. يدعم هذا البرنامج محاكاة سلسلة شرائح STM32 ويتكامل مع أدوات تحليل افتراضية مثل أجهزة القياس ومذبذبات الرسم، ويتم استخدامه على نطاق واسع في الدورات الإلكترونية في المؤسسات العليا. ومع ذلك، يتبنى Proteus نموذج ترخيص تجاري بتكلفة عالية، وتكون سرعة المحاكاة مقيدة بدقة مستوى الدائرة، وينخفض كفاءة التنفيذ بشكل واضح مع زيادة حجم النظام.

على المستوى المحلي، أجرت جامعات Tsinghua و Zhejiang وغيرها من المؤسسات
العملية استكشافات استباقية حول بيئات التجارب الافتراضية المدمجة.
بنت بعض المؤسسات منصات تجارب مدمجة عن بُعد قائمة على الويب، حيث
يمكن للطلاب الوصول إلى المعتاد الحقيقي في غرف الكمبيوتر وترغب
المتصفح لإكمال تحميل الشيفرة والتصحيح عبر الإنترنت. على الرغم من أن
هذه الحلول حققت المشاركة عبر الشبكة لموارد المعتاد، إلا أنها لا تزال مرتبطة
بالأجهزة المادية في المستوى الأساسي، ولم تتمكن من إزالة تكاليف شراء
المعتاد وقيود عدد الأجهزة من الجذور.

في السنوات الأخيرة، ارتفع معدل اختراق لغة Python في البحث العلمي والتدريس
بأسرعة، وبدأت منصات التدريس المدمجة القائمة على المحاكاة البرمجية
الصرافة بلغة عالية المستوى في دخول مجال رؤية الباحثين. تستخدم هذه
الحلول Python لوصف سلوك نواة MCU والمكونات، مع مزايا تشمل دورة تطوير
قصيرة وقدرة توسيع مريحة وتوافق عبر أنظمة التشغيل، ولكنها لا تزال
متخلفة عن المحاكيات ذات المستوى الأدنى من حيث دقة التوقيت وكفاءة
التشغيل.

1.2.2 البحث في أنظمة المراقبة البيئية

على مستوى الحساسات وأجهزة المراقبة، طرحت الشركات المصنعة الدولية العديد
من الأجهزة من المنتجات التجارية. تشكّل سلسلة SHT للحرارة والرطوبة من
شركة Sensirion الأمريكية، وسلسلة BMP للضغط الجوي من شركة Bosch الألمانية،
وسلسلة CCS لحساسات الغاز من شركة AMS النمساوية، تمثيلاً للمستويات
المتقدمة في الصناعة. تتميز هذه المنتجات عمومًا بدقة فحص عالية واستهلاك
طاقة تشغيل منخفضة وواجهات رقمية موحدة، وتستخدم على نطاق واسع في
أجهزة المراقبة البيئية الاحترازية.

حققت المؤسسات البحثية المحلية تقدماً ملحوظاً أيضاً في هذا الاتجاه.
أجرت الأكاديمية الصينية للعلوم وجامعة Tsinghua وغيرها من الوحدات البحثية
دراسات منهجية حول مراقبة الملوثات الجوية واختبار جودة المياه وقياس معدلات
التربة، وشكلت عدداً من أجهزة المراقبة ذات حقوق الملكية الفكرية المستقلة.
في سوق المستهلك، تتكامل أجهزة كشف البيئية الذكية المنزلية التي تطرحها
شركات Huawei و Xiaomi وغيرها عادةً عن أضرار استشعار متعددة الأنواع ووحدات اتصال
Wi-Fi حيث يمكن للمستخدمين الاستعلام عن مؤشرات البيئية في الوقت الفعلي
عبر التطبيقات المحمولة، وتكون قبول السوق مرتفعاً.

فيما يتعلق بالمنصات السحابية، توفر ThingSpeak و Adafruit IO في الخارج،
و OneNET ومنصة IoT Cloud Alibaba في الداخل، بنية تحتية نسبية كاملة لتخزين
البيانات المرصودة عن بُعد وتحليلها الحساسة والعرض الرسومي. تستخدم
هذه المنصات عمومًا بروتوكول MQTT كوسيلة وصول للطرفيات، وتدمر محرك
قواعد وآليات إشعار الأحداث.

من الجدير بالملاحظة أن نتائج البحث والمنصات التجارية الحالية موجهة
بشكل أساسي لسيناريوهات النشر الحقيقي، وتفتقر إلى الاهتمام الكافي

بالقوانين المعرفية والاحتياجات التجريبية الخاصة بعملية التدريب. لا يزال كيفية تجريد ومحاكاة سير عمل تطوير أنظمة المراقبة البيئية لتكون متوافقة مع أهداف التدريب في الدورات المدمجة موضوعاً يحتاج إلى مناقشة معمقة.

1.3 الأعمال الرئيسية للورقة البحثية

تتناول هذه الورقة البحثية موضوع المحاكاة الافتراضية لمحطة مراقبة بيئية ذكية مبنية على STM32 وتنطلق من الأبعاد الأربعة التالية:

(1) **تصميم وتنفيذ منصة عتاد STM32 الافتراضية.** باستخدام STM32F103C8T6 كنموذج أولي، تم تصميم منصة MCU افتراضية كاملة، تشمل محاكاة نواة Cortex-M3 ونظام تعيين الذاكرة ومؤقت SysTick وGPIO للإدخال/الإخراج العام وADC لمحوّل التماثل إلى الرقمة ووحدة تحكم I2C bus وUART للاتصال التسلسلي ومؤقت TIM العام وغيرها من الوحدات الأساسية. تتميز هذه المنصة بالتنفيذ بلغة Python، مع مزايا تشمل قابلية قراءة الشيفرة العالية وسهولة التوسع والتشغيل عبر الأنظمة الأساسية.

(2) **تطوير برامج تشغيل الحساسات البيئية الافتراضية.** بناءً على المنصة العتادية الافتراضية، تم تطوير برامج تشغيل افتراضية لخمس أنواع شائعة من الحساسات البيئية: DHT11 وMQ-135 وBH1750 وBMP280 وحساس الضواء. تم إنشاء إطار موحد لنمذجة الحساسات، يشمل تعريف معلمات البيئية ونموذج تحويل الكميات الفيزيائية ونموذج الضوضاء الغاوسي، مما يجعل البيئات المحاكاة تتبع القوانين الفيزيائية وتتمتع بعشوائية معينة، أقرب إلى خصائص إخراج الحساسات الحقيقية.

(3) **بناء نظام تصورات بيانات الويب.** تم تطوير خدمة الواجهة الخلوية باستخدام إطار عمل Python، Flask واستخدم قاعدة بيانات SQLite لتخزين بيانات الحساسات وسجلات التنبيه وسجلات النظام، مع تصميم واجهة برمجية تطبيق RESTful للاستدعاء من قبل الواجهة الأمامية. تستخدم الواجهة الأمامية تقنيات HTML/CSS/JavaScript الأصلية، وتدمج مكتبة الرسوم البيانية Chart.js لتحقيق عرض الخطوط الزمنية في الوقت الفعلي للبيانات والمعلومات الإحصائية ولوحة التنبيه.

(4) **آلية التحكم في معلمات البيئية والتفاعل.** تم تصميم آلية اتصال بين العمليات (IPC) قائمة على مشاركة الملفات، لتحقيق الاتصال ثنائي الاتجاه بين لوحة تحكم الويب وMCU الافتراضي عبر ملف env_control.txt. يمكن للمستخدمين تعديل معلمات البيئية في الوقت الفعلي (مثل درجة الحرارة والرطوبة والضغط الجوي) عبر واجهة الويب، وتقوم الحساسات الافتراضية بتوليد بيانات محاكاة مقابلة وفقاً للمعلومات المحدثة، مما يحقق تفاعلاً مغلق الحلقة.

1.4 هيكل الورقة البحثية

يتألف النص الكامل من سبعة فصول، ويتم تنظيم محتويات كل فصل على النحو التالي:

الفصل 1 هو المقدمة، ويقدّم خلفية البحث وأهميته، والحالة الحالية للبحث في الداخل والخارج، والأعمال الرئيسية للورقة البحثية، وهيكل الورقة البحثية. الفصل 2 هو التصميم العام للنظام، ويحلل متطلبات الوظائف ومتطلبات الأداء للنظام، ويصمم الهندسة المعمارية العامة ذات الأربع طبقات، ويقدم اختصار التقنيات الرئيسية.

الفصل 3 هو تصميم منصة عتاد STM32 الافتراضية، ويشرح بالتفصيل مبادئ المحاكاة وطرق التنفيذ لنواة MCU الافتراضية ونظام الذاكرة والملاحظات المختلطة.

الفصل 4 هو برنامج تشغيل الحساسات وجمع البيانات، ويقدم الإطار العام لنموذج الحساسات الافتراضية، ويحلل بالتفصيل مبادئ محاكاة كل حساس وتنفيذ برنامج التشغيل.

الفصل 5 هو تصميم النظام البرمجي وتنفيذه، بما في ذلك تصميم وتنفيذ وحدة تخزين البيانات ونظام التنبيه ونظام تصور الويب ولوحة التحكم.

الفصل 6 هو اختبار النظام والتحقق منه، ويغطي اختبار الوظائف واختبار الأداء والتحقق من دقة الحساسات، ويقدم تحليل نتائج الاختبار.

الفصل 7 هو الخلاصة والآفاق، ويُلخّص الأعمال الرئيسية لهذه الورقة البحثية، ويقدم اتجاهات التحسين المستقبليّة.

2 التصميم العام للنظام

2.1 تحليل متطلبات الوظائف

2.1.1 المتطلبات الوظيفية

يحتاج هذا النظام إلى تنفيذ منصة افتراضية لمحطة مراقبة بيئية ذكية، وتشمل أهدافه الوظيفية الجوانب الستة التالية:

(1) **محاكاة MCU الافتراضية:** يجب أن يكون النظام قادراً على محاكاة الوظائف الأساسية للتحكم الدقيق، STM32F103C8T6 بما في ذلك تنفيذ تعليمات المعالج والوصول إلى الذاكرة ونظام الساعة والتحكم في الملحقات. يجب أن يتمكن الشيفرة المدمجة المكتوبة بلغة C (أو الشيفرة المكتوبة بلغة Python) من العمل بشكل صحيح على المنصة الافتراضية دون تعديل قبل النقل إلى العتاد الحقيقي.

(2) **جمع بيانات متعدد الحساسات:** يجب أن يدعم النظام جمع بيانات من خمسة حساسات بيئية على الأقل، بما في ذلك درجة الحرارة والرطوبة وجودة الهواء (المعبر عنها بتركيز الغاز الضار) وشدة الإضاءة والضغط الجوي ومستوى الضوضاء. يجب أن يكون لكل حساس برنامج تشغيل مستقر، ويتبع بروتوكول الاتصال المقابل.

(3) **تخزين وإدارة البيانات:** يجب أن يكون النظام قادراً على تخزين بيانات الحساسات المجمعة بشكل دائم في قاعدة البيانات، مع دعم الاستعلام حسب

الناطق الزمني والتحليل الإحصائي وتصدير البيانات. في الوقت نفسه، يجب تسجيل سجلات تشغيل النظام وأحداث التنبيه لتسهيل استكشاف الأخطاء والتدقيق.

(4) **تصور البيانات في الوقت الفعلي:** يجب أن يوفر النظام واجهة عرض بيانات في الوقت الفعلي عبر الويب، تعرض القراءات الحالية والاتجاهات التاريخية لكل حساس بشكل بديهي من خلال بطاقات رقمية ورسوم بيانية خطية. يجب أن تدعم الرسوم البيانية وظائف تكبير المحاور الزمني وتلميح نقاط البيانات. (5) **وظيفة التنبيه الذكي:** يجب أن يدعم النظام تعيين حدود عليا وسفلى لكل حساس، وعندما تتجاوز قراءة الحساس النطاق المحدد، يتم تشغيل التنبيه تلقائياً وتسجيل معالومات التنبيه في قاعدة البيانات ولوحة الواجهة الأمامية. يجب أن تكون حالة التنبيه قابلة للتحديث والمسح في الوقت الفعلي. (6) **التحكم في معلمات البيئية:** يجب أن يسمح النظام للمستخدمين بتعديل معلمات البيئية في الوقت الفعلي عبر واجهة الويب، لمحاكاة سلوك الحساسات في ظروف بيئية مختلفة. يجب أن تصبح أوامر التحكم فعالة في الوقت المناسب، وتظهر قيم إعدادات البيئية الحالية على الواجهة.

2.1.2 المتطلبات غير الوظيفية

بالإضافة إلى الأهداف الوظيفية، يجب أن تتوافق تشغيل النظام مع الشروط التالية:

(1) **متطلبات الأداء:** يجب أن تكون كفاءة تنفيذ تعليمات MCU الافتراضية عالية بما يكفي، ولا يتجاوز وقت معالجة دورة جمع بيانات واحدة ثانية واحدة. يجب التحكم في وقت استجابة واجهة برمجة تطبيقات الويب في حدود 100 مللي ثانية، وتكون تردد تحديث الرسوم البيانية للواجهة الأمامية لا يقل عن 1 هرتز.

(2) **متطلبات الموثوقية:** يجب أن يكون النظام قادراً على العمل بشكل مستمر وContinuous لمدة 72 ساعة على الأقل دون انقطاع، ويجب أن تكون عمليات قاعدة البيانات مزودة بآلية حماية المعاملات لمنع فقدان البيانات أو التناقض. يجب أن تكون برامج تشغيل الحساسات قادرة على التعامل مع حالات الاستثناء مثل انتهاء مهلة الاتصال وأخطاء التحقق.

(3) **متطلبات القابلية للتوسع:** يجب أن يتمتع النظام بتصميم معماري معياري جيد، وعند إضافة أنواع جديدة من الحساسات أو وحدات الملحقات، يجب أن يتحقق ذلك عن طريق إضافة ملفات التشغيل وملفات التكوين دون تعديل الشيفرة الأساسية. يجب أن يتبع تصميم واجهة برمجة تطبيقات الواجهة الخلوية للويب مواصفات RESTful لتسهيل توسيع وظائف الواجهة الأمامية.

(4) **متطلبات سهولة الاستخدام:** يجب أن يوفر النظام وثائق وإرشادات استخدام واضحة، ويجب أن تكون واجهة الويب منسقة بشكل مقبول وبديهية في التشغيل. يجب أن تكون واجهة برمجة تطبيقات الاعداد الافتراضي مصممة بشكل موجز، لتقليل تكلفة التعلم وصعوبة التطوير للمستخدمين.

(5) **متطلبات التوافق عبر الأنظمة الأساسية:** يجب أن يكون النظام قادراً

على العمل على أنظمة التشغيل الرئيسية مثل Windows و Linux و macOS، دون الاعتقاد على خصائص عتاد أو نظام تشغيل محدد. يجب أن يتم تنفيذ المنصة الافتراضية نفسها باستخدام لغة Python واستخدام خصائص Python عبر الأنظمة الأساسية لضمان التوافق.

2.2 التصميم المعماري العام

بناءً على المتطلبات المذكورة أعلاه، يتم تقسيم النظام من الأسفل إلى الأعلى إلى أربع طبقات: طبقة تجريد العتاد، وطبقة برامج تشغيل الأجهزة، وطبقة منطق الأعمال، وطبقة العرض والتطبيقات. يوضح الجدول 1 حدود وظائف كل طبقة.

جدول 1: تصميم الهندسة المعمارية ذات الأربع طبقات للنظام

وصف المسؤولية	الاسم	الطبقة
مسؤولية عن عرض واجهة الويب أمامية ورسم الرسوم البيانية والتفاعل مع المستخدم وعرض البيانات، باستخدام HTML/CSS/JavaScript الأصلي	التطبيقات والعرض	4 الطبقة
مسؤولية عن خدمة الواجهة الخلفية Flask وتصميم API RESTful وإدارة تخزين البيانات ومعالجة منطق التنبيه وتحليل أوامر التحكم	الأعمال منطق	3 الطبقة
مسؤولية عن تطوير برامج تشغيل الحساسات المختلطة وتنفيذ دورة جمع البيانات، وتحويل الكميات الفيزيائية إلى كميات رقمية	الأجهزة تشغيل برامج	2 الطبقة
مسؤولية عن محاكاة نواة MCU الافتراضية ونظام الذاكرة ووحدات التحكم في bus والملحقات المختلطة، وتوفير واجهة على مستوى مسجلات العتاد	العتاد تجريد	1 الطبقة

تشكل طبقة تجريد العتاد (HAL Layer, Abstraction (Hardware أساس الكمال للمنصة، حيث يتسلسلوكها على مستوى المسجلات مع شريحة STM32 الحقيقية. تستدعي طبقة برامج تشغيل الأجهزة الواجهات المكشوفة بواسطة HAL لإكمال تهيئة الحساسات وقراءة البيانات وتحليل البروتوكولات. تتحمل طبقة منطق الأعمال مهام معالجة البيانات الخام والتخزين الدائم والتحليل، وتفتح القدرات

للطبقة العليا على شكل واجهات برمجة تطبيقات موحدة. تركز طبقة العرض
التطبيقي على تحسين جودة العرض التفاعلي.
يتعاون الطبقات من خلال عقود واجهات موحدة، ويؤمن الوصول المباشر للطبقات
العليا إلى الحالة الداخلية للطبقات السفلى دون المرور بالواجهات. يتحكم
هذا التصميم بشكل فعال في درجة الاقتران بين الوحدات. عند الحاجة إلى إدخال
حساس جديد، يكفي إضافة ملف التشغيل المقابل في طبقة برامج تشغيل
الأجهزة وتسجيل قواعد التحليل في طبقة منطق الأعمال، دون الحاجة إلى تعديل
طبقة تجريد العتال أو طبقة العرض التطبيقي.

2.3 اختيار التقنيّة

يتبع النظام في اختيار الأدوات والإطارات معيار "البساطة والكفاءة وسهولة
التوسع"، ويوضح الجدول 2 المجموعة الأساسية للتقنيات.

جدول 2: اختيار التقنيات للنظام

سبب الاختيار	التقنية	الفئة
بناء جملة موجز وكفاءة تطوير عالية ومكتبات قياسية وخارجية غنية ودعم جيد عبر الأنظمة الأساسية	Python 3.11	التطوير لغة
خفيف الوزن ومرن وسهل التعلم ومجتمعات نشطة وإضافات غنية	Flask 3.0	الويب عمل إطار
تكوين صغري وتخزين ملف واحد ولا يتطلب عملية خادم مستقلة، مناسبة للتطبيقات الصغيرة والمتوسطة	SQLite 3	البيانات قاعدة
لا يعتمد على بناء أدوات وسرعة تحميل سريعة وتوافق جيد وسهولة النشر	الأصلي HTML/CSS/JS	الأمامية الواجهة تقنيات
مجانية ومفتوحة المصدر وواجهة برمجة تطبيقات ودية ودعم التصميم المتجاوب والرسوم المتحركة	Chart.js 4.x	البيانية الرسوم مكتبة
إدارة إصدار موزعة وإدارة فروع مرنة وكفاءة تعاون عالية	Git	الإصدارات في التحكم

يتم اختيار Python 3.11 كلغة تطوير أساسية بناءً على ثلاثة اعتبارات:
أولاً، قواعد بناء جملة Python بسيطة وقابلة لقراءة الشيفرة بارزة، مما يسهل
الشرح السطر بسطر في السيناريوهات التشغيلية؛ ثانياً، تغطي المكتبة

القياسية المهمة الشائعة مثل الاتصال بالشبكة والوصول إلى قواعد البيانات وعمليات الملفات والبرمجة المتزامنة، مما يقلل من إدخال التبعية الخارجية؛ ثالثاً، يزيل نمط التنفيذ التفسيري انتظار التجميع، ويمكن التحقق من الصحة فوراً بعد التعديل، مما يختصر دورة التصحيح. Flask هو إطار عمل ويب خفيف الوزن قائم على Python وفلسفته تؤكد على البساطة والمرونة. بالمقارنة مع إطارات العمل الكاملة مثل Django لا يفرض Flask بنية دليل مشروع محددة مسبقاً ولا علاقات ربط بالملفات، ويمكن للمطورين اختصار قاعدة البيانات ومحرك القوالب واستراتيجيات المصادقة حسب الاحتياجات الفعلية. بالنسبة لتطبيق ويب بهذا الحجم المعتدل، تكون درجة تغطية وظائف Flask في نطاق معقول، وتتجنب التصميم المفرط وفي الوقت نفسه لا تسبب نقصاً في القدرات.

تعد SQLite قاعدة بيانات علائقية مدمجة، حيث يتم تجميع جميع البيانات في ملف قرص واحد، ولا تتطلب عملية خدمة مستقلة. تبسط هذه الخاصية "الاستخدام الفوري" بشكل كبير عملية النشر، وهي مناسبة بشكل خاص للتطوير الفردي والتجارب التشغيلية والسيناريوهات الصغرى الحجم. يدعم SQLite بناء جملة SQL القياسي ومعاملات ACID، وهو ما يكفي لدعم متطلبات التخزين الدائم وتحليل الاستعلامات في هذا النظام. اختارت استراتيجيات تقنيات الواجهة الأمامية HTML5 و CSS3 و JavaScript الأصلية، دون إدخال إطارات مكونات مثل React أو Vue. الحافز الأساسي لهذا القرار هو التحكم في التعقيد الإجمالي، وتتجنب عبء تكوين سلسلة أدوات البناء، والحفاظ على شكل شيفرة الواجهة الأمامية قابلاً للقراءة المباشرة. تقدم Chart.js كمكتبة مكونات رسوم بيانية مفتوحة المصدر مجموعة متنوعة من فئات الرسوم البيانية وأبعاد التكوين، ويظهر محرك الرسم القائم على Canvas أداءً جيداً من حيث الأداء، ويمكنه تلبيات المتطلبات الأداء للصور البيانية في الوقت الفعلي.

3 تصميم منصة عتاد STM32 الافتراضية

تشكل منصة عتاد STM32 الافتراضية الركيزة الأساسية للنظام بأكمله. تكمن فكرة بناء هذه المنصة في إعادة إنتاج سلوك STM32F103C8T6 على مستوى المسجلات في بيئية برمجية صرفية، بحيث يمكن لبرامج تشغيل الحساسات وشيفرة طبقة التطبيق العمل كما لو كانت على الشريحة الحقيقية. فيما يلي، يتم تقديم الشرح التفصيلي لمحاكاة نواة MCU وآلية تعيين الذاكرة والمحاكاة الرئيسية للملحقات.

3.1 تصميم نواة MCU الافتراضية

3.1.1 نظرة عامة على معلمات STM32F103C8T6

تنتج STM32F103C8T6 من شركة STMicroelectronics، وهي مزودة بنواة Cortex-ARM M3، وتنتهي إلى طراز الكثافة المتوسطة في سلسلة STM32F1. حققت هذه الشريحة توازناً جيداً بين الأداء والتكلفة، وبالتالي تم تبنيها على نطاق واسع في التعليم الجامعي والممارسة الهندسية. يوضح الجدول 3 المؤشرات التقنية الرئيسية لها.

جدول 3: المؤشرات التقنية الرئيسية لـ STM32F103C8T6

المواصفة	المعلمة عنصراً
ARM Cortex-M3	النواة بنوية
ميغاهرتز 72	رئيسي تردد أقصى
كيلوبايت 64	Flash ذاكرة
كيلوبايت 20	SRAM
LQFP48 (تغلي ف 37)	GPIO دبابيس
قناة 16 بت، 12 ذو ADC محول 2	ADC
أساسية مؤقتات 2 + متقدم مؤقتات 1 + عامة مؤقتات 3	المؤقتات
CAN و SPI 1 و USART 2 و I2C 2	الاتصال واجهات
2.0 V – 3.6 V	التشغيل جهد
–40°C – +85°C	التشغيل حرارة درجة
LQFP48	التغلي ف نوع

نظراً لأن هذه المنصة تستخدم Python للتنفيذ، ولم يتم إجراء محاكاة دورية تعليمة بتعليمة لبنية مجموعة تعليمات ARM أو دورات الساعة، بل تم التركيز على إعادة إنتاج سلوك مسجلات الملحقات بشكل واقعي. يعتمد مقياس الزمن الأساسي على 1 مللي ثانية كأصغر وحدة، مع الاستعانة بمكتبة time في Python لإكمال التأخير والتوقي.

3.1.2 محاكاة مؤقت SysTick

تدمج نواة Cortex-M3 مؤقت SysTick، وهو عداد تنازلي ذو 24 بت، يوفر إيقاعاً دورياً لنظام التشغيل والتطبيقات. في المنصة الافتراضية المنفذة بلغة Python، تركز محاكاة SysTick على ثلاث قدرات: تقليل العداد وتحديث قيمة إعادة التحميل وتشغيل المقاطعة. بسبب خصائص تنفيذ اللغة التفسيرية، يصعب ضمان دقة التوقيات على مستوى الميكروثانية، وبالتالي يتم اعتماد مخططات قريبي قائم على الوقت النسبي.

تتحمّل دالة delay_ms مسؤولية التأخير على مستوى الميلي ثانية. تسجل هذه الدالة وقت النظام الحالي عند نقطة الدخول، ثم تستطلع وتتحقق من كمية

الوقت المنقضي، وتعود فوراً بمجرد الوصول إلى عدد الميثللي ثانية المحدد. ستستراتيجيات الانتظار للنشطة تشغيل وحدة المعالجة المركزية، لكنها كافية للعرض التشغيلي والتحقق من الوظائف.

```
emit tropmi | 1
| 2
:)sm(sm_yaled fed | 3
| 4
" " " | 5
# دالة تأخير رب الميثللي ثانية
# المعاملات: | 6
sm # عدد الميثللي ثانية لل تأخير | 7
" " " | 8
)(emit.emit = trats | 9
:sm < 0001 * )trats - )(emit.emit( elihw | 10
ssap | 11
```

Listing 1: تنفيذ دالة msdelay

تُظهر الشيفرة 1 تنفيذ delay_ms باستراتيجيات الانتظار للنشطة. يمكن تنظيم مناقشة بين الطالب حول إيجابيات وسلبيات هذا المخطط أثناء التدريس: البساطة في التنفيذ وعدم وجود تبعيات خارجية هي نقاط القوة؛ وعدم قدرة وحدة المعالجة المركزية على التعامل مع مهام أخرى خلال فترة التأخير هو نقطة ضعف واضحة. إذا كان التحسين مطلوباً، يمكن محاولة إعادة كتابته باستخدام threading أو asyncio إلى نسخة غير حاجبة.

3.2 تصميم نظام الذاكرة

3.2.1 نموذج تعيين الذاكرة

يقوم STM32F103C8T6 بتوحيد العنونة لذاكرة Flash و SRAM ومسجلات الملحقات في مساحة عنوان 32 بت. وفقاً لهذا، قام نظام الذاكرة الافتراضي بإعادة إنتاج ثلاث مناطق عنوان رئيسية:

(1) **ذاكرة Flash**: تُستخدم لتخزين شيفرة البرنامج وبيانات الثوابت، ويتراوح نطاق العنوان من 0x08000000 بحجم 64 كيلوبايت. في المنصة الافتراضية، تُستخدم منطقة Flash بشكل أساسي لتخزين شيفرة البرنامج الثابتة المحاكاة ومعلومات التكوين.

(2) **SRAM**: تُستخدم لتخزين المتغيرات وقت التشغيل وذاكرة المكدس والذاكرة الديناميكية، ويتراوح نطاق العنوان من 0x20000000 بحجم 20 كيلوبايت. يتم تنفيذ منطقة SRAM في المنصة الافتراضية على شكل مصفوفة بايت، مع دعم عمليات القراءة والكتابة على مستوى البايت ونصف الكلمة والكلمة.

(3) **مسجلات الملحقات**: تشمل مسجلات التكوين ومسجلات البيانات للملحقات مثل GPIO و ADC و UART و TIM و I2C، ويتراوح نطاق العنوان من 0x40000000. يشغل

كل ملحق جزءاً متجاوراً من مساحة العنوان، ويتم ترتيب المسجلات حسب الإزاحات الثابتة.

تُعد فئة MemoryRegion جوهر نظام الذاكرة الافتراضية، حيث تدير جزءاً متجاوراً من مساحة العنوان لقراءة وكتابة تدفق البايت. يتبنى STM32 نمط Little-Endian، حيث يتم وضع البايت المنخفض للبيانات متعددة البايت في العنوان المنخفض والبايت العالي في العنوان المرتفع، ويتبع MemoryRegion بدقة قواعد ترتيب البايت هذه عند القراءة والكتابة.

3.2.2 تنفيذه نمط Little-Endian

يعني نمط Little-Endian أنه: يتم تخزين بايت البيانات العالي في العنوان المرتفع، وبايت البيانات المنخفض في العنوان المنخفض. على سبيل المثال، بالنسبة لـ 0x12345678، يكون ترتيب البايت في الذاكرة على التوالي 0x78 و 0x56 و 0x34 و 0x12، مع زيادة العنوان من المنخفض إلى المرتفع.

توفر مكتبة struct في Python وسائل التحكم في ترتيب البايت: رمز التوسيق '<' يعني Little-Endian، و'>' يعني Big-Endian. تستخدم فئة MemoryRegion struct.pack و struct.unpack لإكمال تسلسل وإلغاء تسلسل البيانات متعددة البايت، وبالتالي ضمان تناسق معالجة ترتيب البايت بين المنصة الافتراضية و STM32 الحقيقي.

3.2.3 تعيين المسجلات

يقوم RegisterMap بتغليفي إضافي فوق MemoryRegion موجهاً نحو إدارة مسجلات الملحقات. يحتوي كل مثيل ملحق على كائن RegisterMap، ويمكن للمطورين الوصول عبر اسم المسجل بدلاً من العنوان العاري، وبالتالي تحسين قابلية قراءة الشيفرة.

توفر RegisterMap أربع قدرات رئيسية:

- (1) **تسجيل المسجلات:** عند الانتهاء، يتم تعريف جميع المسجلات التي يملكها هذا الملحق، بما في ذلك اسم المسجل والعنوان الإزاحي والقيمة الافتراضية وأذونات الوصول (للكتابة فقط/للكتابة فقط/للكتابة).
- (2) **الوصول إلى حقول البت:** يدعم عمليات القراءة والكتابة لبت واحد أو حقل بت في المسجل. على سبيل المثال، يمكن تعيين البت 0 في مسجل CR1 إلى 1 عبر set_bit('CR1', 0)، دون الحاجة إلى قراءة المسجل بأكمله ثم الحساب اليدوي.
- (3) **حماية الكتابة:** بالنسبة للمسجلات للكتابة فقط (مثل مسجل الحالة SR)، سيرفض RegisterMap عمليات الكتابة المباشرة، مما يمنع الشيفرة العلوية من تعديل قيم المسجلات التي لا يجب تعديلها عن طريق الخطأ.
- (4) **رد الاتصال عند التغيير:** يسمح بتسجيل دوال رد اتصال لمسجلات محددة، وتشغيل رد الاتصال تلقائياً عندما تتغير قيمة المسجل. تعد هذه الآلية مهمة جداً لمحاكاة استجابة سلوك الملحقات، على سبيل المثال، عند كتابة مسجل ODR الخاص بـ GPIO، يجب تحديث حالة إخراج الدبوس فوراً.

3.3 محاكاة المداخلات

3.3.1 GPIO للمدخلات/الإخراج العام

تتحمّل GPIO مسؤولية إدخال وإخراج الإشارات الرقمية، وهي البوابة الأولى لتفاعل المتحكم الدقيق مع الدوائر الخارجية. يحتوي منفذ GPIO في STM32F103C8T6 على 16 دبوساً لكل مجموعة Pin (0 إلى 15) ويمكن تكوين كل دبوس بشكل مستقر كإدخال عائم أو إخراج سحب لأعلى أو إدخال سحب لأسفل أو إدخال تناظري أو إخراج دافع/مجمع مفتوح.

تتمحور محاكاة GPIO حول الآليات التالية:

- (1) **مصفوفة حالة الدبابيس:** يحافظ كل منفذ GPIO على مصفوفة بطول 16، تسجل حالة الجهد الحالية لكل دبوس (0 أو 1) بالنسبة لوضع الإدخال، يتم تحديد حالة الدبوس بواسطة الجهاز الخارجي أو مقاومات السحب لأعلى/لأسفل؛ بالنسبة لوضع الإخراج، يتم تحديد حالة الدبوس بواسطة قيمة مسجل ODR.
- (2) **مسجل بيانات الإدخال IDR:** يُستخدم (Input IDR) Register لقراءة مستوى الجهد الحالي لكل دبوس في منفذ GPIO في المنصة الافتراضية، يتم الحصول على قيمة IDR عن طريق ترميز مصفوفة حالة الدبابيس. على وجه التحديد، يكون البت i في IDR مساوياً لحالة الجهد الحالية للدبوس رقم i . يتم حساب IDR وفقاً للمعادلة:

$$(1) \quad IDR = \sum_{i=0}^{15} P_i \times 2^i$$

حيث P_i تمثل حالة الجهد للدبوس رقم i ، و $P_i \in \{0, 1\}$.

- (3) **مسجل بيانات الإخراج ODR:** يُستخدم (Output ODR) Register لتعريف مستوى الجهد الخارجي لكل دبوس في منفذ GPIO عند كتابة قيمة جديدة في ODR، يقوم وحدة GPIO الافتراضية بتحويل البتات، وتحديث حالة الإخراج المقابلة للدبوس، ويشغل دوال رد الاتصال المسجلة.

- (4) **آلية رد الاتصال:** لدعم استجابة الأجهزة الخارجية لتغيرات حالة GPIO، تم تنفيذ آلية تسجيل رد اتصال في GPIO الافتراضي. يمكن للمستخدمين تسجيل دوال رد اتصال لأحداث الحافة الصاعدة أو الحافة الهابطة أو تغيير المستوى للدبوس محدد. عند حدوث التغيير المقابل في حالة الدبوس، يتم تنفيذ جميع دوال رد الاتصال المسجلة بالترتيب. تعد هذه الآلية مفيدة للغايات عند محاكاة إخراج المقاطعة للحساسات والتحكم في تشغيل/إطفاء مؤشرات LED.

3.3.2 ADC لمحوّل التماثل إلى الرقمي

يقوم ADC بتعريف الجهد التماثلي المستمر إلى كمية رقمية منفصلة، ويلعب دور المترجم بين الواجهة التماثلية والواجهة الرقمية. يدمج STM32F103C8T6 مساري ADC من نوع SAR ذو 12 بت، ويدعم 16 قناة خارجية وقناتين داخليتين (حساس حرارة داخلي وجهد مرجعي).

تشمل الخصائص الرئيسية لـ ADC الافتراضي ما يلي:
(1) **دقة 12 بت:** نتيجة تحويل ADC هي عدد صحيح بدون إشارة ذو 12 بت، ويتراوح النطاق من 0 إلى 4095. تكون صيغة التكميم المقابلة:

$$(2) \quad D = \left\lfloor \frac{V_{in}}{V_{ref}} \times 4096 \right\rfloor$$

حيث D هو الإخراج الرقمي، و V_{in} هو الجهد التماثلي المدخل، و V_{ref} هو الجهد المرجعي (عادةً 3.3V).

(2) **التحويل من الجهد إلى الكممية الرقمية:** عندما تقوم الشيفرة العليا بقراءة قناة ما عبر ADC، يقوم ADC الافتراضي بحساب الكممية الرقمية المقابلة وفقاً لقيمة الجهد التماثلي الحالية المتصلة بالقناة وبناءً على المعادلة 2. يتم توفير قيمة الجهد التماثلي بواسطة الحساس الافتراضي المتصل أو مصدر الإشارة الخارج.

(3) **نموذج الضوضاء:** لمحاكاة خصائص تحويل ADC بشكل أكثر واقعية، يتم تراكب ضوضاء بيضاء غاوسية على نتيجة التحويل المثالية. يمكن تعديل سرعة الضوضاء عبر معاملات التكوين، ويتم تعيّن القيمة الافتراضية إلى ± 2 (Least Significant Bit). Significant (Least LSB ± 2 Bit) تكون صيغة التحويل مع الضوضاء:

$$(3) \quad D_{actual} = D_{ideal} + N(0, \sigma^2)$$

حيث $N(0, \sigma^2)$ تمثل عدداً عشوائياً موزعاً بشكل طبيعي بمتوسط 0 وتباين σ^2 ، وقيمة σ الافتراضية حوالي 2.

(4) **اختيار القناة والتمكين:** يدعم ADC الافتراضي تكوين قنوات متعددة، ويحدد مسجل تسلسل (Regular SQR Sequence (Register) ترتيب القنوات وعددها. قبل إجراء التحويل، يجب تمكين ADC أولاً (تعيّن بت ADON في مسجل CR2 إلى 1)، ثم بدء التحويل (تعيّن بت SWSTART في مسجل CR2 إلى 1).

3.3.3 وحدة تحكم I2C bus

يكمل I2C bus الاتصال متعدد الرؤوس ومتعدد التتابع باستخدام خطين SDA و SCL، ويستخدم على نطاق واسع في الحساسات وذاكرات EEPROM وغيرها من الأجهزة. يجهز STM32F103C8T6 بمجموعتين من واجهات I2C، متوافقة مع الوضع القياسي (100 كيلوهرتز) والوضع السريع (400 كيلوهرتز).

تركز محاكاة I2C bus الافتراضية على السلوكيات التالية:

(1) **تركيب الجهاز:** يحافظ I2C bus الافتراضي على قائمة بالأجهزة المُركّبة، ويتم تحديدها لكل جهاز بعنوان تابع فريد من 7 بت. عندما ترسل الشيفرة العليا إشارة بدء وبت العنوان عبر I2C، يبحث وحدة تحكم bus عن مثيل الجهاز الافتراضي المقابل وفقاً للعنوان، وينشئ اتصالاً.

يتم تعريف واجهة تركيب الجهاز على النحو التالي:

1 | fed hcatta _ ecived fles , sserdda , ecived :

وتشغيلها تلقائيًا عند وصول البيانات. تحاكي هذه الآلية مقاطعة الاستقبال في UART الحقيقي، مما يُمكّن الشيفرة العلوية من معالجة بيانات المنفذ التسلسلي بطريقة مدفوعة بالأحداث، دون الحاجة إلى استطلاع المخزن المؤقت للاستقبال.

```

1  :TRAUlauriV ssalc
2  :)fles(__tini__ fed
3  002511 = etarduab.fles
4  enoN = kcablac_xr.fles
5  ][ = reffub_xt.fles
6
7  :)kcablac ,fles(kcablac_xr_tes fed
8  """ تسجل لدالة رداتصال الاستقبال """
9  kcablac = kcablac_xr.fles
10
11 :)atad ,fles(dnes fed
12 """ إرسال البيانات تسلسلية ( أوباي ) """
13 :)rts ,atad(ecnatsnisi fi
14 )'8-ftu'(edocne.atad = atad
15 )atad(dnetxe.reffub_xt.fles
16 :kcablac_xr.fles fi
17 )atad(kcablac_xr.fles
18
19 :)sgra* ,tmf ,fles(ftnirp fed
20 """ftnirp إخراج منسق، محاكاة"""
21 sgra % tmf = gsm
22 )gsm(dnes.fles

```

Listing 3: تسجيل رد اتصال UART الافتراضي وإرسال

تُظهر الشيفرة 3 تنفيذ فئة VirtualUART، حيث تدعم واجهة send كلاً من الأنواع str و bytes، وتعيد printf دلالات الإخراج المنسق في لغة C.

3.3.5 TIM للمؤقت العام

يُعد TIM الأكثر تنوعاً في عائلة ملحقات STM32، حيث يمكنه العمل كمؤقت أو عداد أو توليد موجة PWM أو التقاط الإدخال الخارجي. يستخدم هذا المشروع فقط قدرته الأساسية على التوقيت، كمصدر تحفيز بتردد ثابت لأخذ عينات الحساسات. يستخدم VirtualTIM خيط حماية (daemon thread) من مكتبة threading في Python لإكمال التوقيت في الخلفية. ينتهي خيط الحماية تلقائياً عند خروج الخيط الرئيسي، دون الحاجة إلى التنظيف اليدوي، وهو مناسب تماماً لاحتياجات المهام الدورية في الخلفية.

يمكن تقسيم منطق تشغيل TIM الافتراضي إلى ثلاث خطوات:
(1) زيادة العداد: بعد تشغيل TIM، يزداد الخيط الحماية العداد الداخلي بفرات

زمنية ثابتة (تحدد معامل التقسيم المسبق وقيمة إعادة التحميل التلقائي).
تكون صيغة حساب تردد زيادة العداد:

$$(4) \quad f_{count} = \frac{f_{clk}}{(PSC + 1) \times (ARR + 1)}$$

حيث f_{clk} هو تردد ساعة المؤقت (عادةً 72 ميجاهرتز)، و PSC هو قيمة المقسم المسبق،
و ARR هو قيمة إعادة التحميل التلقائي.

(2) **رد الاتصال الدوري:** عندما يصل العداد إلى قيمة إعادة التحميل التلقائي،
يتم تصفير العداد وتشغيل حدث التحديث (Update). (Event) إذا تم تسجيل دالة
رد الاتصال الدوري، يتم استدعاؤها في هذا الوقت. في نظام المراقبة البيئية،
تقوم دالة رد الاتصال الدوري عادةً بتنفيذ مهمة جمع بيانات الحساسات، وبالتالي
تحقيق أخذ العينات التلقائي بتردد ثابت.

(3) **التشغيل والإيقاف:** يتحكم TIM في التشغيل والإيقاف عبر بت CEN
في مسجل CR1. تعيّن بت CEN إلى 1 يبدأ المؤقت، وتصفيره يوقف المؤقت.
يقوم TIM الافتراضي بإنشاء وبدء خيط الحماية عند التشغيل، ويضبط علامة
حدث لإخطار الخيط بالخرج عند الإيقاف.

```

gnidaerht tropmi | 1
emit tropmi | 2
| 3
:MITlautriV ssalc | 4
:)fles(__tini__ fed | 5
CSP # 9917 = relacserp.fles | 6
RRA # 9999 = daoler_otua.fles | 7
0 = retnuoc.fles | 8
eslaF = gninnur.fles | 9
enoN = kcabllac_doirep.fles | 10
)(tnevE.gnidaerht = tneve_pots_.fles | 11
| 12
:)kcabllac ,fles(kcabllac_doirep_tes fed | 13
"" #"" تعيّن دالة رد الاتصال الدوري | 14
kcabllac = kcabllac_doirep.fles | 15
| 16
:)fles(daerht_remit_ fed | 17
"" #"" خيط حماية المؤقت | 18
/ )1 + daoler_otua.fles( * )1 + relacserp.fles( = sm_doirep | 19
00027
:)(tes_si.tneve_pots_.fles ton elihw | 20
)0.0001 / sm_doirep(peels.emit | 21
0 = retnuoc.fles | 22
:kcabllac_doirep.fles fi | 23

```

```

24 | ) (kcabllac_doirep.fles
25 |
26 | :)fles(trats fed
27 |     "" # "" بدء المؤقت
28 |     eurT = gninnur.fles
29 | ) (raelc.tneve_pots_.fles
30 | ) eurT=nomead ,daerht_remit_.fles=tegrat(daerhT.gnidaerht = t
31 | ) (trats.t
32 |
33 | :)fles(pots fed
34 |     "" # "" إيقاف المؤقت
35 |     eslaF = gninnur.fles
36 | ) (tes.tneve_pots_.fles

```

Listing 4: تنفيذ خيط الحماية للمؤقت الافتراضي TIM

تُظهر الشيفرة 4 كيف يحقق VirtualTIM التشغيل الدوري بمساعدة خيط الحماية. يتم تنفيذ `period_callback` عن كل تجاوز للدورة، وتعديل `prescaler` و `auto_reload` لإعادة تعيين فترة التوقيت، وبالتالي مطابقة معدلات أخذ عينات متنوعة.

4 برامج تشغيل الحساسات وجمع البيانات

تُعتبر الحساسات بمثابة أعضاء الحس في نظام المراقبة، حيث تحول المعلومات المختلطة في البيئية الفيزيائية إلى إشارات كهربائية لمعالجتها بواسطة MCU. في بيئة المحاكاة البرمجية الصرفة، تلعب نماذج الحساسات دور الوصل بين المعتاد الافتراضي والتطبيقات العلوي، حيث يؤثر درجة واقعيته بشكل مباشر على مصداقية النظام بأكمله. فيما يلي، يتم مناقشة فكرة النمذجة العامة أولاً، ثم تحليل مبادئ المحاكاة وترميز برامج التشغيل لكل حساس على حدة.

4.1 مبادئ المحاكاة الافتراضية للحساسات

4.1.1 إطار النمذجة العام

أنشأ هذا النظام إطار نمذجة موحد لمحاكاة الحساسات، حيث يتم اختزال جميع أنواع الحساسات إلى ثلاث مراحل: إدخال معلومات البيئية ونموذج تحويل الكميات الفيزيائية **والإخراج الرقمي**. يوضح الشكل 1 المخطط المفاهيمي لهذا الإطار.

إطار نمذجة الحساسات الافتراضية

شكل 1: المخطط المفاهيمي لإطار نمذجة الحساسات الافتراضية

يحتوي هذا الإطار على ثلاثة عناصر جوهرية:

(1) **تعريف معلمات البيئية:** يرتبط كل حساس بمجموعة من متغيرات معلمات البيئية، والتي تمثل القيم الحقيقية للكميات الفيزيائية التي يقيسها الحساس. على سبيل المثال، يرتبط حساس درجة الحرارة بمعامل $temperature$ ، ويرتبط حساس الإضاءة بمعامل $illuminance$. يمكن تعديل قيم معلمات البيئية في الوقت الفعلي عبر لوحة التحكم، لمحاكاة ظروف بيئية مختلفة.

(2) **نموذج تحويل الكميات الفيزيائية:** يصف كيفية تحويل معلمات البيئية إلى الكميات الفيزيائية المخزنة بواسطة الحساس (عادةً جهد أو مقاومة أو كمية رقمية). تختلف نماذج التحويل للحساسات المختلفة، وقد تكون علاقة خطية أو علاقة أسية أو علاقة دالة قطعية. يجب إنشاء نماذج التحويل بناءً على أوراق بيانات الحساسات (Datasheet) والمبادئ الفيزيائية، لضمان معقولية البيانات المحاكاة.

(3) **نموذج الضوضاء:** يتضمن إخراج الحساسات الحقيقية ضوضاء عشوائية لا مفر منها، وتأتي مصادر الضوضاء من الضوضاء الحرارية وضوضاء الطلقة والتداخلات الكهرومغناطيسية وغيرها. لتعزيق واقعية المحاكاة، يتم تراكب ضوضاء بيضاء غاوسية على الإخراج المثالي للحساس. التعبير الرياضي لنموذج الضوضاء هو:

$$Y = X + N(0, \sigma^2) \quad (5)$$

حيث Y هي القيمة الفعلية للإخراج الحساس، و X هي القيمة المثالية (المحسوبة بواسطة نموذج تحويل الكميات الفيزيائية)، و $N(0, \sigma^2)$ هي الضوضاء العشوائية الموزعة بشكل طبيعي بمتوسط 0 وتباين σ^2 .

يتم تحديد قيمة شدة الضوضاء σ بناءً على مستوى دقة الحساس وسيناريوهات التطبيق النموذجية. بالنسبة للحساسات عالية الدقة (مثل BMP280) تأخذ σ قيمة صغيرة؛ بالنسبة للحساسات منخفضة الدقة (مثل MQ-135) تأخذ σ قيمة أكبر. في السيناريوهات الخاصة، يمكن أيضاً إضافة قيم شاذة (Outlier) لمحاكاة أعطال الحساس أو التغيرات المفاجئة في البيئية.

4.1.2 واجهة برنامج تشغيل الحساس

تشارك برنامج تشغيل الحساسات الافتراضية المختلطة في نفس عقد الواجهة، حيث لا تحتاج الشيفرة العليا إلى الاهتمام بالاختلافات الأساسية عند الاستدعاء. يتم تعريف الواجهة على النحو التالي:

- `init()`: تهيئة الحساس، وتكوين معاملات الاتصال ووضع التشغيل.
 - `read()`: قراءة القيمة الحالية المقاسة للحساس، وإرجاع البيانات الخام أو الكمية الفيزيائية المعيارية.
 - `calibrate()`: تنفيذ عملية المعايرة، وتحديث معاملات المعايرة (إن وجدت).
 - `set_environment(env_dict)`: تحديث معلومات البيئية المرتبطة بالحساس.
- بفضل اتساق الواجهة، عند إدخال نوع جديد من الحساسات، يكفي كتابة فئة برنامج التشغيل المقابلة وتنفيذ الأربع طرق، بينما تظل دورة جمع البيانات وطبقة عرض الويب دون تغيير.

4.2 حساس DHT11 للحرارة والرطوبة

4.2.1 نظرة عامة على الحساس

يدمج DHT11 استشعار درجة الحرارة والرطوبة في نفس الشريحة، ويتواصل مع المضيف عبر بروتوكول bus أحادي. تتميز بانخفاض التكلفة والحجم الصغير وسهولة البدء، مما يجعلها تظهر بشكل متكرر في مشاريع إنترنت الأشياء للمبتدئين. [يوضح الجدول 4](#) مواصفات الرئسيية.

جدول 4: المؤشرات التقنية للرئسيية لـ DHT11

المواصفة	المعلمة عنصر
20% – 90% RH	(الرطوبة) الرقياس نطاق
±5% RH	(الرطوبة) الرقياس دقة
0°C – 50°C	(درجة الحرارة) الرقياس نطاق
±2°C	(درجة الحرارة) الرقياس دقة
1% RH / 1°C	الدقة
ثانية 1 ≥	العينات أخذ دورة
3.3 V – 5.5 V	التشغيل جهد
تسلسلي أحادي bus	الاتصال بروتوكول

4.2.2 محاكاة بروتوكول bus أحادي

يمكن تقسيم اتصال bus أحادي لـ DHT11 إلى ثلاث مراحل: بدء المضيف، وإرسال الاستجابة بواسطة الحساس، ونقل البيانات بت بت.

(1) **بدء المضيّف:** يسحب MCU خط البيانات إلى منخفض لمدة لا تقل عن 18 مللي ثانية، ثم يحرره وينتظر استجابة DHT11. يتم استخداً هذا الإجراء لإيقاظ الحساس في حالة الاستعداد، وإعلامه بأنه على وشك إرسال البيانات.

(2) **استجابة الحساس:** بعد اكتشاف نبضة البدء، تُخرج DHT11 أولاً مستوى منخفضاً لمدة 80 ميكروثانية، ثم مستوى عالٍ لمدة 80 ميكروثانية، لإعلام المضيّف بأنها جاهزة. يقوم المضيّف بتأكيد وجود الحساس عبر التقاط تسلسل النبضات هذا.

(3) **نقل إطار البيانات:** تُبلغ DHT11 عن درجة الحرارة والرطوبة بتنسيق إطار 40 بت. يبدأ كل بت بمستوى منخفض لمدة 50 ميكروثانية، ثم يحدد عرض المستوى العالي القيمي للمنطقية: 26-28 ميكروثانية تمثل "0" وحوالي 70 ميكروثانية تمثل "1". يتم ترتيب 40 بت بالترتيب: 8 بت رطوبة صحيحة و8 بت رطوبة عشرية و8 بت درجة حرارة صحيحة و8 بت درجة حرارة عشرية و8 بت مجموع تحقق.

يُعدي برنامج تشغيل VirtualDHT11 إنتاج توقيات البروتوكول المذكور أعلاه بشكل كامل، والتنفيذ الرئيسي على النحو التالي:

```

1  :11THDlautriV ssalc
2  :)nip ,oipg ,fles(__tini__ fed
3  oipg = oipg.fles
4  nip = nip.fles
5  %05 # الرطوبة الافتراضية ytidimuh.fles = 0.05
6  C52 # درجة الحرارة الافتراضية erutarepmet.fles = 0.52
7
8  :)vne ,fles(tnemnorivne_tes fed
9  )0.05 , 'ytidimuh' (teg.vne = ytidimuh.fles
10 )0.52 , 'erutarepmet' (teg.vne = erutarepmet.fles
11
12 :)fles(daer fed
13 # محاكاة إشارة البدء
14 ) 'TUPTUO' ,nip.fles(edom_tes.oipg.fles
15 )0 ,nip.fles(etirw.oipg.fles
16 )02(sm_yaled
17 )1 ,nip.fles(etirw.oipg.fles
18 )03(su_yaled
19 ) 'TUPNI' ,nip.fles(edom_tes.oipg.fles
20
21 # محاكاة إشارة الاستجابة ونقل البيانات
22 # إرجاع الرطوبة , درجة الحرارة )
23 ytidimuh.fles(tni = tni_h
24 )01 * )tni_h - ytidimuh.fles((tni = ced_h
25 )erutarepmet.fles(tni = tni_t

```

<pre>)01 *)tni_t - erutarepmet.fles((tni = ced_t FFx0 &)ced_t + tni_t + ced_h + tni_h(= muskcehc </pre>	26 27 28
<pre>]muskcehc ,ced_t ,tni_t ,ced_h ,tni_h[= atad \ ,)5.1 ,0(ssuag.modnar + ytidimuh.fles nruter)8.0 ,0(ssuag.modnar + erutarepmet.fles </pre>	29 30 31

Listing 5: إشارة بدء DHT11 وقراءة البيانات

في الشيفرة 5، يعيد VirtualDHT11 إنتاج توقيات bus أحادي عبر قلب مستوى GPIO. تقوم مرحلة توليد البيانات بتعبئة معلمات البيئية في إطار 40 بت مع إلحاق مجموع التحقق، ثم تُدخل ضوابط غاوسية وفقاً للمعادلة 5: يتم تعيين الانحراف المعياري لقناة الرطوبة إلى 1.5% RH، وقناة درجة الحرارة إلى 0.8°C مما يتوافق مع مستوى دقة DHT11 الحقيقي.

4.3 حساس MQ-135 لجودة الهواء

4.3.1 مبدأ الحساس

يظهر MQ-135 حساسية عالية لأمونيا والكبريتيد وبخار البنزين والدخان وغيرها من الغازات. تستخدم الطبقة الحساسة في لم أكسيد القصدير (SnO₂) شبه الموصل، حيث تحدث تفاعلات الأكسدة والاختزال لجزيئات الغاز المستهدف على سطحه، مما يؤدي إلى تغيير في مقاومة الحجم، وبالتالي انعكاس تركيز الغاز.

توجد علاقة أسية بين مقاومة العنصر الحساس R_S وتركيز الغاز C :

$$(6) \quad \frac{R_S}{R_0} = A \cdot C^{-B}$$

حيث R_0 هي قيمة المقاومة المرجعية للحساس في الهواء النظيف، و A و B هما ثابتان تجريبيان مرتبطان بنوع الغاز، و C هو تركيز الغاز (بوحدة ppm بالنسبة لسيناريوهات مراقبة جودة الهواء العامة، يتم عادةً أخذ $A \approx 116.6$ و $B \approx 2.77$).

4.3.2 الاستدلال العكسي من الجهد إلى التركيز

يحول دائرة الجهد المقسم داخل وحدة MQ-135 تغيير المقاومة إلى إشارة جهد. يفي جهد الإخراج بما يلي:

$$(7) \quad V_{out} = V_{cc} \cdot \frac{R_L}{R_S + R_L}$$

حيث V_{cc} هو جهد التشغيل (عادةً 5V)، و R_L هو مقاومة الحمل (مقياس الجهد القابل للتعدّل).

بناءً على هذا، يمكن حل مقاومة العنصر الحساس:

$$(8) \quad R_S = R_L \cdot \left(\frac{V_{cc}}{V_{out}} - 1 \right)$$

بتعويض المعادلة أعلاه في المعادلة 6، نحصل على تعبير تحليلي لتركيز الغاز:

$$(9) \quad C = \left(\frac{A \cdot R_0}{R_S} \right)^{\frac{1}{B}} = \left(\frac{A \cdot R_0 \cdot V_{out}}{R_L \cdot (V_{cc} - V_{out})} \right)^{\frac{1}{B}}$$

4.3.3 تنفيذ برنامج التشغيل الافتراضي

يأخذ برنامج تشغيل VirtualMQ135 معامل البيئية air_quality (بوحدة ppm) كمُدخل، ويستدل عكسيًا على جهد ADC وفقاً للمعادلات 6 و 9. بالإضافة إلى ذلك، يعيد البرنامج التشغيل أيضاً ثلاثة سلوكيات نموذجية للجهاز الحقيقي:

(1) **محاكاة التسخين المسبق:** يحتاج حساس MQ-135 إلى فترة تسخين مسبق بعد التشغيل للوصول إلى حالة تشغيل مستقرة، وتكون فترة التسخين المسبق النموذجية 20 ثانية. يسجل برنامج التشغيل الافتراضي وقت بدء التشغيل بعد التهيئة، ويعيد بيانات غير صالحة أو بيانات ذات دقة منخفضة خلال فترة التسخين المسبق.

(2) **الاستجابة غير الخطية:** وفقاً للعلاقة الأسية، تكون العلاقة بين تركيز الغاز والمقاومة علاقة غير خطية. في منطقة التركيز المنخفض، تكون حساسية الحساس أعلى؛ في منطقة التركيز المرتفع، تنخفض الحساسية تدريجياً. يعيد برنامج التشغيل الافتراضي محاكاة هذه الخاصية غير الخطية بشكل كامل.

(3) **تعويض درجة الحرارة والرطوبة:** يتأثر إخراج MQ-135 بدرجة الحرارة والرطوبة البيئية. يُدخل برنامج التشغيل الافتراضي معامل تعويض بسيط لدرجة الحرارة والرطوبة، ويصحح قيمة التركيز وفقاً لدرجة الحرارة والرطوبة الحالية، مما يحسن من واقعية البيانات المحاكاة.

4.4 حساس BH1750 للإضاءة

4.4.1 نظرة عامة على الحساس

يُخرج BH1750 كمية رقمية بشكل مباشر بوحدة lux عبر I2C، bus مما يلغي الحاجة إلى ADC خارجي والمعالجة المعقدة. يتميز باتساع النطاق وارتفاع الدقة وبساطة الأسلاك، ويُستخدم على نطاق واسع في سيناريوهات الإضاءة التلقائية وتعديل الإضاءة الخلفية والكشف البيئي.

يحدد بايت الأوامر وضع القياس في BH1750، ويوضح الجدول 5 الأوامر الشائعة.

جدول 5: أوامر I2C الشائعة لـ BH1750

وصف الوظيفة	الاسم	الأمر قيـمة
أمر التشغيل، إيقاف الحساس	Power On	0x01
أمر إيقاف التشغيل، الدخول في وضع استهلاك طاقة منخفض	Power Down	0x00
وضع الدقة العالية المستمر (دقة 1 lux، وقت قياس 120 مللي ثانية)	Continuous H-Res Mode	0x10
وضع الدقة العالية المستمر 2 (دقة 0.5 lux، وقت قياس 120 مللي ثانية)	Continuous H-Res2 Mode	0x11
وضع الدقة المنخفضة المستمر (دقة 4 lux، وقت قياس 16 مللي ثانية)	Continuous L-Res Mode	0x13
وضع الدقة العالية لمرة واحدة	One Time H-Res Mode	0x20

4.4.2 صيغة حساب الإضاءة

في وضع H-Res، يُخرج BH1750 عدداً أولياً ذو 16 بت (raw value)، ثم يتم ضربه في معامل حساسية 1.2 lux/عداد للحصول على الإضاءة الفعلية:

$$E_v = \text{raw} \times 1.2 \quad (10)$$

حيث E_v هي الإضاءة (بوحدة lux) و raw هي البيانات الأولية المُرجّعة من الحساس. إذا كانت قيمة raw 500، فإن الإضاءة المقابلة هي $500 \times 1.2 = 600$ lux. بعد استلام الأمر عبر I2C، يحسب VirtualBH1750 العداد الأولي عكسياً وفقاً لمعامل البيئية illuminance الحالي وبناءً على المعادلة 10:

$$\text{raw} = \left\lfloor \frac{E_v}{1.2} \right\rfloor + N(0, \sigma^2) \quad (11)$$

حيث يتم أخذ σ 3 وحدات عداد، مما يتوافق مع ضوضاء قياس حوالي 3.6 lux، وهو متوافق مع مواصفات ورقة بيانات BH1750.

4.5 حساس BMP280 للضغط الجوي

4.5.1 نظرة عامة على الحساس

يأتي BMP280 من Bosch، وهو مبنى على عملية MEMS، ويتميز بحجم صغير واستهلاك طاقة منخفض ودقة عالية. يمكن رؤيته في محطات الطقس وأجهزة الارتفاع والملاحة الداخلية والتحكم في الدوران للطائرات بدون طيار وغيرها من المجالات. يوضح الجدول 6 المؤشرات التقنينة الرئيسية.

جدول 6: المؤشرات التقنية الرئيسية لـ BMP280

المواصفة	المعلمة عن صر
300 hPa – 1100 hPa	الضغط قياس نطاق
±0.12 hPa (قيمة)	الضغط قياس دقة
–40°C – +85°C	الحرارة درجة قياس نطاق
±0.5°C (عند 25°C)	الحرارة درجة قياس دقة
±1 hPa	المطلقة الدقة
I2C أو SPI	الاتصال واجهة
0x58	الشرعية معرف

4.5.2 التحقق من معرف الشرية

يوجد لدى BMP280 مسجل معرف شرية للقراءة فقط عند العنوان 0xD0 وتكون قيمته 0x58 بعد إعادة التعيين. عادةً ما يقوم المضيف بقراءة هذا المسجل أولاً بعد التشغيل لتأكيد وجود الجهاز. يقوم VirtualBMP280 بتضمين هذا السلوك في مرحلة التهيئة، حيث يُرجع 0x58 بشكل ثابت لقراءة العنوان 0xD0.

4.5.3 ADC ذو 24 بت ومعاملات المعايرة

يدمج BMP280 ADC ذو 24 بت، ويقوم بتكسيم الإخراج التماثلي لحساسات الضغط ودرجة الحرارة على التوالي. يكون عدد البتات الفعالة لكل من قناتي الضغط ودرجة الحرارة 20 بت، ويتم تخزينها في مسجلات 3 بايت. تتيح الدقة العالية جداً التقاط تقلبات الضغط على مستوى الميليبار. يكتب BMP280 الحقيقي معاملات التعويض في ذاكرة NVM (العنوان 0x88–0xA1) أثناء المعايرة في المصنع. يجب على المضيف استخراج هذه المعاملات قبل القراءة، ثم تشغيل خوارزمية تعويض درجة الحرارة التي تحتوي على حدود من الدرجة الثانية والثالثة في جانب MCU لاستعادة الكمية الفيزيائية من قيمة ADC الأولية.

قامت المنصة الافتراضية بتبسيط عملية التعويض بشكل معقول، واستخدمت علاقة خطية لتقريب التعيين بين الضغط ودرجة الحرارة والقيمة الأولية لـ ADC:

$$(12) \quad \text{raw_P} = \left\lfloor \frac{P}{1013.25} \times 415148 \right\rfloor + N(0, \sigma_P^2)$$

$$(13) \quad \text{raw_T} = \lfloor (T + 40) \times 100 \rfloor + N(0, \sigma_T^2)$$

حيث P هي قيمة الضغط الجوي البيئي (بوحدة hPa) و T هي قيمة درجة الحرارة البيئية (بوحدة °C) و 415148 هي قيمة ADC الأولية النموذجية المقابلة لـ 1013.25 hPa (الضغط الجوي القياسي)، و σ_P و σ_T هما الانحرافان المعياريان للضوضاء. استناداً إلى حقيقتة تجريبية: في وضع التجاوز الزائدي $\times 16$ تكون القيمة الأولية النموذجية المقابلة لـ 1013.25 hPa حوالي 415148. على الرغم من أن علاقة النسبة بين الضغط والقيمة الأولية التي تم إنشاؤها بهذه الطريقة تتجاهل

التعويض المعقد لدرجة الحرارة والتصحيح غير الخطي في الشريحة الحقيقية، إلا أنه لا تزال تتمتع بدقة كافية للعرض التعلليمي والتحقق الروتيي.

4.6 حساس الضوضاء

4.6.1 مبدأ الحساس

يقيس حساس الصوت (شائع مثل وحدة KY-038 أو المستندة إلى LM393) شدة الضوضاء البيئية، ويعرض النتيجة بالديسيبل (dB). تتكون هذه الوحدات عادةً من ميكروفون إلكتروني حساس للغاية ودائرة معالجة الإشارة، حيث تحول موجات الضغط الصوتي إلى جهد تماثلي. الديسيبل هو وحدة نسبية لوغاريتمية. يتم تعريف مستوى الضغط الصوتي (Sound SPL) Level على النحو التالي:

$$(14) \quad L_p = 20 \cdot \log_{10} \left(\frac{p}{p_0} \right)$$

حيث L_p هو مستوى الضغط الصوتي (بوحدة dB) و p هو الضغط الصوتي الفعلي (بوحدة Pa)، و p_0 هو الضغط الصوتي المرجعي (عادةً 20×10^{-6} Pa، أي حد السمع البشري عند تردد 1 كيلوهرتز).

4.6.2 التعديلي الخطي للجهد

بعد التضخيم والتقويم، يكون إخراج وحدة الحساس متناسباً مع الضغط الصوتي. في النموذج الافتراضي، يتم افتراض علاقة خطية إضافية بين الجهد ومستوى الضغط الصوتي:

$$(15) \quad V_{out} = V_{offset} + k \cdot L_p$$

حيث V_{offset} هو جهد الإزاحة (المقابل للإخراج عند 0 dB، وعادةً ما يكون قيمة موجبة صغيرة في الواقع)، و k هو معامل الحساسية (بوحدة V/dB). يقرأ برنامج تشغيل VirtualNoise معامل البيئية noise_level (بوحدة dB) ويحسب جهد الإخراج وفقاً للمعادلة 15، ثم يقوم ADC بتكميمه إلى قيمة رقمية. للالتزام أكثر من السيناريوهات الحقيقية، يتم تراكب ضوضاء غاوسية بانحراف معياري 1.5 dB على القراءة النهائية، وهو ما يعادل الدقة النموذجية لمقياس مستوى الصوت الاستهلاكي.

يغيد VirtualNoise أيضاً المحاكاة التالية لثلاث خصائص فيزيائية:

(1) **التوجيهية:** يمثل الميكروفون الحقيقي توجيهية معينة، حيث تختلِف حساسيته للصوت القادم من اتجاهات مختلفة. يُدخل الحساس الافتراضي معامل توجيهي بسيط لمحاكاة هذه الخاصية الفيزيائية.

(2) **الاستجابة الترددية:** تختلِف حساسية الأذن البشرية للصوت بترددات مختلفة، ويُعد التوزيع A (A-weighting) الطريقة الأكثر استخداماً لتوزيع مستوى الضغط الصوتي. يُخرج الحساس الافتراضي بشكل افتراضي مستوى الضغط الصوتي الموزع وفقاً للتوزيع A، وهو أقرب إلى الحساس الذاتي للأذن البشرية.

(3) **الاحتفاظ بالقيمة:** تدعم بعض حساسات الضوضاء وظيفة كشف القيمة، حيث تسجل أقصى مستوى للضغط الصوتي خلال فترة زمنية. يقوم الحساس الافتراضي بتنفيذ وضع الاحتفاظ بالقيمة الاختياري، حيث يسجل ويعيد أقصى قيمة dB خلال نافذة زمنية محددة.

4.7 تصميم دورة جمع البيانات

تقود دورة جمع البيانات التدفق الرئيسي لمحطة المراقبة الافتراضية بأكملها: تستدعي الحساسات لإكمال أخذ العينات بالدورة الثابتة، ثم تدفع النتائج إلى سلسلة التخزين والعرض. يتم تقسيم دورة واحدة إلى 6 مراحل:

1. **قراءة معلمات البيئية:** قراءة إعدادات معلمات البيئية الحالية من الملف المشترك `env_control.txt`، بما في ذلك درجة الحرارة والرطوبة والضغط الجوي والإضاءة وجودة الهواء ومستوى الضوضاء.

2. **تحديث بيئية الحساسات:** تمرير معلمات البيئية المقروءة إلى مثيلات الحساسات الافتراضية المختلطة، بحيث يمكن للحساسات توليد بيانات محاكاة وفقاً لأحدث إعدادات البيئية.

3. **تنفيذ قراءة الحساسات:** استدعاء طريقة `read()` لكل حساس على التوالي للحصول على القيمة المقاسة الحالية. يكون ترتيب القراءة عادةً: DHT11 (درجة الحرارة والرطوبة) → BMP280 (الضغط/درجة الحرارة) → BH1750 (الإضاءة) → MQ-135 (جودة الهواء) → حساس الضوضاء.

4. **تنسيق البيانات:** تحويل القراءات الأولية للحساسات إلى وحدات قياسية (مثل lux و hPa و ppm و dB و غيرها)، وتنظيمها في شكل قاموس أو تنسيق JSON وفقاً لهيكل البيانات المعروف مسبقاً.

5. **تخزين البيانات:** إدراج سجل البيانات المُنسق في جدول `sensor_data` في قاعدة بيانات SQLite، وتحديث ذاكرة التخزين المؤقت للبيانات الأحدث في الذاكرة في الوقت نفسه.

6. **فحص التنبيه:** المرور على تنبيهات التنبيه لكل حساس، والتحقق مما إذا كانت القراءة الحالية تتجاوز نطاق الحدود. إذا تجاوزت، يتم تسجيل سجل التنبيه وتحديث حالة التنبيه.

لتجنب استهلاك وقت طويل لأخذ العينة في دورة واحدة مما يؤدي إلى اهتزاز الدورة، يُدخل النظام استراتيجيات تعويض الدورة. ليكن الدورة المستهدفة T_{sample} (بالثانية)، والوقت الفعلي المستغرق في هذه الدورة $t_{elapsed}$ ، ثم يتم حساب وقت السكون على النحو التالي:

$$(16) \quad t_{sleep} = \max(0, T_{sample} - t_{elapsed})$$

تضمن المعادلة 16 أن الدورة الفعلية لا تقل عن القيمة المحددة. إذا انتهت مهلة أخذ العينة في دورة واحدة بالفعل، يتم تخطي السكون والدخول مباشرة إلى الدورة التالية، مع إعطاء الأولوية لضمان حدث البيانات.

```

emit tropmi | 1
| 2
DOIREP_ELPMAS = 0.2 # دورة أخذ العينات 2 ثانية | 3
| 4
:)gifnoc_mralla ,bd ,srosnes(pool_noitcelloc_atad fed | 5
| 6
| 7
:eurT elihw
| 8
)(emit.emit = trats_t
| 9
# الخطوة 1: قراءة معلمات البيئية | 10
)(lortnoc_tnemnorivne_daer = vne | 11
| 12
# الخطوة 2: تحدي ثبيئية الحساسات | 13
:srosnes ni rosnes rof | 14
)vne(tnemnorivne_tes.rosnes | 15
| 16
# الخطوة 3: تنفيذ قراءة الحساسات | 17
){ = sgnidaer | 18
:srosnes ni rosnes rof | 19
))(daer.rosnes(etadpu.sgnidaer | 20
| 21
# الخطوة 4: تنسيق البيانات | 22
)sgnidaer(atad_rosnes_tamrof = drocer | 23
| 24
# الخطوة 5: تخزين البيانات | 25
)drocer(atad_rosnes_tresni.bd | 26
| 27
# الخطوة 6: فحص التنبيه | 28
)bd ,gifnoc_mralla ,drocer(smrala_kcehc | 29
| 30
# تعويض الدورة | 31
trats_t - )(emit.emit = despale_t | 32
)despale_t - DOIREP_ELPMAS ,0(xam = peels_t | 33
)peels_t(peels.emit | 34

```

Listing 6: التنفيذ الأساسي لدورة جمع البيانات

تُظهر الشيفرة 6 التنفيذ الكامل لـ data_collection_loop الذي حدّد ثابت SAMPLE_PERIOD

دورة أخذ العينات، ويكبح منطق تعويض الدورة انحراف الوقت. يحمل قاموس readings
النتائج الوسطى بين الخطوات، ويتم إدخال السجل المُنسق في النهاية إلى
قاعدة البيانات والمشاركة في حكم التنبيه.

5 تصميم النظام البرمجي وتنفيذه

يشكل المعتاد الافتراضي STM32 وبرامج تشغيل الحساسات الأساس التشغيلي
السفلي، ويعمل أربع وحدات جوهرية - وحدة تخزين البيانات ونظام التنبيه
ونظام تصور الويب ولوحة التحكم - بشكل تعاوني فوق هذا الأساس، وتتحمّل
على التوالي مسؤوليات تخزين البيانات بشكل دائم والكشف عن الحالات
الشاذة وعرض النتائج والتحكم عن بُعد، مما يشكل معاً تطبيق برمجي كاملاً
لمحطة المراقبة البيئية الذكية.

5.1 وحدة تخزين البيانات

يجب على نظام المراقبة أولاً كتابة قراءات الحساسات المجمعة وحالة التشغيل
وأحداث التنبيه في وسيط تخزين دائم، قبل أن يمكن إجراء المتتبع التاريخي
والتحليل الإحصائي وتدقيق السجلات. باعتباره محرك قاعدة بيانات النظام،
لا تتطلب SQLite عملية خدمة مستقلة، وتخزن قاعدة البيانات بأكملها في
ملف واحد، مما يجعل النشر والترحيل مريحين للغاية.

5.1.1 تصميم هيكل جداول قاعدة البيانات

تحتوي قاعدة البيانات على ثلاثة جداول جوهرية: sensor_data (جدول بيانات الحساسات)
و alarm_log (جدول سجلات التنبيه) و system_log (جدول سجلات النظام).
(1) جدول sensor_data: يُستخدم لتخزين بيانات الاستشعار في الوقت الفعلي
المجمعة من الحساسات المختلطة، وهو الجدول الأكبر حجماً. يوضح الجدول 7 تصميم
هيكله، ويحتوي على 10 حقول.

جدول 7: هيكل جدول sensor_data

الوصف	القيود	نوع البيانات	اسم الحقل
مفتاح أساسي متزايد تلقائياً	PRIMARY AUTOINCREMENT	INTEGER	id
الطابع الزمني للجمع	NOT NULL CURRENT_TIMESTAMP	DATETIME	timestamp
قيمة درجة الحرارة، بوحدة C°		REAL	temperature
قيمة الرطوبة، بوحدة % RH		REAL	humidity
قيمة الضغط الجوي، بوحدة hPa		REAL	pressure
شدة الإضاءة، بوحدة lux		REAL	illuminance
مؤشر جودة الهواء، بوحدة ppm		REAL	air_quality
مستوى الضوضاء، بوحدة dB		REAL	noise_level
درجة حرارة داخلية لـ MCU، بوحدة C°		REAL	cpu_temp
معرف مصدر البيانات	DEFAULT 'simulation'	TEXT	data_source

يتبنى id استراتيجيات المفاتيح الأساسية المتزايدة تلقائياً، وتخصيص معرف فريد لكل سجل مكتوب؛ يتم ملء timestamp تلقائياً بواسطة قاعدة البيانات عند الإدراج، مما يوفر مرجعاً زمنياً للاسترجاع حسب الفترة الزمنية لاحقاً؛ تعكس الحقول من temperature إلى noise_level ستة كميات فيزيائية بيئية بالترتيب؛ يحفظ cpu_temp قراءة محاكاة حساس الحرارة الداخلية لـ MCU الافتراضي؛ يُعَلَّم data_source بشكل افتراضي على أنه 'simulation' ويحتفظ هذا الحقل بواجهة توسيع للوصول إلى العتاد الحقيقي في المستقبل.

(2) **جدول alarm_log**: يتتبع دورة الحياة الكاملة للتنبيه من الإنشاء إلى الإلغاء. تعني الحقول على النحو التالي: id هو المفتاح الأساسي؛ timestamp يحدد اللحظة الأولى لتشغيل التنبيه؛ sensor_type يحدد الحساس الذي أنتج التنبيه؛ alarm_type يميز بين نوعي التنبيه HIGH و LOW؛ threshold يحدد الحدود المحددة عند التشغيل؛ actual_value يحفظ قراءة الحساس في لحظة التشغيل؛ يُعَلَّم على حالة التنبيه الحالية ACTIVE أو RESOLVED؛ resolved_at يكتب الطابع الزمني للإلغاء التنبيه.

(3) **جدول system_log**: يلتقط الأحداث الرئيسية والمخرجات التصحيحية أثناء تشغيل النظام. الحقول المحددة هي: id مفتاح أساسي؛ timestamp يكتب لحظة إنشاء السجل؛ level يقسم مستويات الخطورة إلى أربع مستويات INFO و DEBUG

وWARNING وERROR؛ module يُغَلِّم على الوحدة الوظيفية التي أنتجت السجل؛ message يحمل محتوى النص المحدد؛ metadata يخزن معلومات منظمه إضافية بتنسيق JSON، مما يسهل التحليل الآلي لاحقاً.

5.1.2 واجهة sqlite3 في Python

توفر مكتبة sqlite3 القياسية المدمجة في Python تغليفاً كاملاً لواجهة C الخاصة بـ SQLite، حيث يمكن للمطورين تنفيذ SQL والتحكم في حدود المعاملات وقراءة مجموعات النتائج مباشرة عليه، مما يوفر خطوات تثبيت برامج تشغيل قواعد البيانات الخارجية.

يتم تغليف كل وصول إلى قاعدة البيانات بواسطة مدير السياق، حيث يتم تحرير الاتصال تلقائياً بعد انتهائه العملية، مما يمنع خطر تسرب المقابس. يتم تعيين journal_mode صراحةً على (Write-Ahead WAL)، (Logging) حيث يمكن لعمليات القراءة والكتابة التقدم بشكل متوازٍ، ويتم تقصير وقت انتظار القفل بشكل كبير عندما يقوم خيط جمع البيانات وخيط خدمة الويب بالوصول إلى نفس قاعدة البيانات في الوقت نفسه.

5.1.3 عمليات CRUD

تكشف وحدة تخزين البيانات عن مجموعة من واجهات CRUD التي تغطي الإنشاء والقراءة والتحديث والحذف:

(1) **Create (إنشاء):** تستقبل insert_sensor_data قراءات الحساسات بشكل قاموس، وتكتبها في جدول sensor_data. تستخدم عبارة الاستعلام ربط المعاملات، حيث لا يمكن للجهات الخارجية العبث بمعنى SQL من خلال بناء مدخلات ضارة، وبالتالي قطع مسار هجمات الحقن.

(2) **Read (قراءة):** توفر الوحدة القدرة على الاسترجاع متعدد الأبعاد: setal_teg (gnidaer_t) تُرجع أحدث نقطة أخذ عينات؛ dne, trats(egnar_yb_sgnidaer_teg) (تسحب جميع السجلات ضمن نافذة زمنية محددة؛ rosneg_yb_sgnidaer_teg) (تسترجع آخر N سجلات تاريخية حسب نوع الحساس).

(3) **Update (تحديث):** تقوم update_alarm_status بتحويل التنبيهات في حالة ACTIVE إلى RESOLVED، وتعني resolved_at إلى الوقت الحالي للنظام، مما يكمل حلقة التنبيه.

(4) **Delete (حذف):** تقوم delete_old_readings(before_date) بمسح السجلات منتهية الصلاحية حسب حدود التاريخ، ويمكن تشغيلها إما بواسطة مهمة مجدولة تلقائياً أو يدوياً بواسطة المسؤول، لتجنب الضغط المستمر على التخزين الناتج عن تراكم البيانات التاريخية.

5.1.4 استعلامات إحصائية

بالإضافة إلى CRUD الأساسية، تدمر الوحدة أيضاً القدرة على التحليل التجميعي، مما يسمح باستخلاص الإحصائيات من بيانات أخذ العينات خلال فترة زمنية. تشمل أبعاد التجميع الشائعة ما يلي:

- (1) **استعلام المتوسط (AVG):** يحسب المتوسط الحسابي لقراءات الحساسات في نافذة زمنية محددة، ويوصف المستوى العام لمعاملات البيئية خلال تلك الفترة.
- (2) **استعلام الحد الأدنى (MIN):** يستخرج أقل قيمة أخذ عينات في تلك الفترة، ويساعد في اكتشاف القيم المنخفضة الشاذة أو الظروف المتطرفة.
- (3) **استعلام الحد الأقصى (MAX):** يستخرج أعلى قيمة أخذ عينات في تلك الفترة، ويساعد في اكتشاف القيم المرتفعة الشاذة أو الظروف المتطرفة.
- (4) **استعلام الانحراف المعياري (STDEV):** يقيس درجة تشتت نقاط أخذ العينات حول المتوسط، وكلما زادت القيمة كانت تقلبات البيئية أكثر حدة. يتم عرض قالب SQL للاستعلام الإحصائي على النحو التالي:

```

TCELES | 1
,pmet_gva sa )erutarepmet(GVA | 2
,pmet_nim sa )erutarepmet(NIM | 3
,pmet_xam sa )erutarepmet(XAM | 4
,ytidimuh_gva sa )ytidimuh(GVA | 5
,erusserp_nim sa )erusserp(NIM | 6
erusserp_xam sa )erusserp(XAM | 7
atad_rosnes MORF | 8
;? DNA ? NEEWTEB pmatsemit EREHW | 9

```

Listing 7: قالب SQL للاستعلام الإحصائي

يستعين قالب SQL أعلاه بدالات التجميع لتلخيص المؤشرات الإحصائية لعدة حساسات دفعة واحدة. يتم تمرير حدود الوقت في عبارة WHERE على شكل معاملات نائبة، مما يحافظ على مرونة الاستعلام ويتجنب أيضاً مخاطر الحقن الانتاجية عن تسلسل السلاسل.

5.2 تصميم نظام التنبيه

يقوم نظام التنبيه بفحص قراءات الحساسات في الوقت الفعلي بشكل مستمر، وبمجرد اكتشاف تجاوز الحدود، يقوم بإنشاء إشعار لتذكير أفراد الصيانة بالتدخل في الوقت المناسب لمعالجة الحالات البيئية الشاذة.

5.2.1 تكوين حدود التنبيه

يحافظ قاموس ALARM_CONFIG على حدود عليا وسفلى مستقلة لكل حساس: high_threshold يحدد الحد الأعلى، و low_threshold يحدد الحد الأدنى. عندما تتجاوز القراءة الحد الأعلى، يتم إنشاء تنبيه من نوع HIGH؛ وعندما تنخفض عن الحد الأدنى، يتم إنشاء تنبيه من نوع LOW، ولا يتداخل النوعان. يتم عرض مثال على تكوين التنبيه الافتراضي على النحو التالي:

```
{ = GIFNOC_MRALA | 1
```

	{ : 'erutarepmet'	2	
C53	# حدالتنبيهبالحرارةالعالية	3	'dlohserht_hgih' : 0.53
C01	# حدالتنبيهبالحرارةالمنخفضة	4	'dlohserht_wol' : 0.01
	, 'C' : 'tinu'	5	
	'درجة الحرارة' : 'eman'	6	
	,	7	
	{ : 'ytidimuh'	8	
%08	# حدالتنبيهبالرطوبةالعالية	9	'dlohserht_hgih' : 0.08
%02	# حدالتنبيهبالرطوبةالمنخفضة	10	'dlohserht_wol' : 0.02
	, 'HR %' : 'tinu'	11	
	'الرطوبة' : 'eman'	12	
	,	13	
	{ : 'erusserp'	14	
aPh 0501	# حدالتنبيهبالضغطالعالي	15	'dlohserht_hgih' : 0.0501
aPh 089	# حدالتنبيهبالضغطالمنخفض	16	'dlohserht_wol' : 0.089
	, 'aPh' : 'tinu'	17	
	'الضغط الجوي' : 'eman'	18	
	,	19	
	{ : 'ytilauq_ria'	20	
mpp 005	# حدالتنبيهبجودةالهواء	21	'dlohserht_hgih' : 0.005
	# لايجوجدحدأدنى	22	'dlohserht_wol' : enoN
	, 'mpp' : 'tinu'	23	
	'جودة الهواء' : 'eman'	24	
	,	25	
	# ... تكوييناتحساساتأخرى	26	
	}	27	

Listing 8: مثال على تكوين حدود التنبيه

كما هو موضح في الشيفرة 8، يتضمن تكوين كل حساس قيم الحدود والوحدات
والأسماء العربية وغيرها. عندما يتم تعيين low_threshold لأحد الحساسات على
None، فهذا يعني عدم مراقبة التنبيه المنخفض، ومراقبة التنبيه المرتفع
فقط.

5.2.2 منطق فحص التنبيه

تقوم دالة check_all بقيادة عملية حكم التنبيه بأكملها: تقرأ تكويينات الحساسات
في ALARM_CONFIG على التوالي، وتقارن قيمة أخذ العينات الحالية بنطاق الحدود،
وتشغل التنبيه أو تلغيه بناءً على نتيجة المقارنة. يتم تقسيم الخطوات
المحددة على النحو التالي:

(1) **الفحص بالتكرار:** استخرج إدخالات الحساسات في ALARM_CONFIG واحداً
تلو الآخر، ومقارنة قيمة أخذ العينات الحالية بنطاق الحدود المحدد لهذا الحساس.

(2) **حكم الحد الأعلى:** عندما تكون قيمة أخذ العينات أعلى من high_threshold، يقوم النظام بالبحث عما إذا كان هناك تنبيه HIGH نشط بالفعل لهذا الحساس: إذا وجد، يتم تحديث actual_value؛ إذا لم يوجد، يتم إدراج سجل تنبيه جديد. ACTIVE.

(3) **حكم الحد الأدنى:** عندما تكون قيمة أخذ العينات أقل من low_threshold، يتم تشغيل تنبيه من نوع LOW وفقاً لنفس الإجراءات الممتثل للحد الأعلى.

(4) **استعادة التنبيه:** بعد أن تعود قيمة أخذ العينات إلى نطاق الحدود، يقوم النظام بتحويل جميع التنبيهات ACTIVE لهذا الحساس دفعة واحدة إلى RESOLVED، ويكتب الطابع الزمني resolved_at.

(5) **التخزين الدائم:** سواء كان تنبيه جديد أو انتقال حالة، يتم استدعاء واجهة قاعدة البيانات على الفور للكتابة على القرص، مما يضمن إمكانية تتبع السلسلة الكاملة للتنبيه.

```

1  :)bd ,gifnoc ,sgnidaer(lla_kcehc fed
2  ""
3  # فحص حالة التنبيه لجميع الحساسات
4  # المعاملات:
5  # :sgnidaer : قاموس قراءات الحساسات الحالية
6  # :gifnoc : قاموس تنبيه
7  # :bd : مثيل عملي لقاعدة البيانات
8  ""
9  (smeti.gifnoc ni gfc ,rosnes rof
10 )rosnes(teg.sgnidaer = eulav
11 :enoN si eulav fi
12 eunitnoc
13
14 # فحص الحد الأعلى
15 )'dlohserht_hgih'(teg.gfc = hgih
16 :hgih > eulav dna enoN ton si hgih fi
17 )eulav ,hgih , 'HGIH' ,rosnes ,bd(mrala_reggirt
18
19 # فحص الحد الأدنى
20 )'dlohserht_wol'(teg.gfc = wol
21 :wol < eulav dna enoN ton si wol fi
22 )eulav ,wol , 'WOL' ,rosnes ,bd(mrala_reggirt
23
24 # فحص استعادة
25 \ dna )hgih =< eulav ro enoN si hgih( fi
26 : )wol => eulav ro enoN si wol(
27 )rosnes ,bd(mrala_evloser

```

Listing 9: المنطق الأساسي لفحص التنبيه

كما هو موضح في الشيفرة 9، تنفيذ دالة `check_all` عملية فحص التنبيه
الكاملة. `trigger_alarm` و `resolve_alarm` هما دالتان مساعدتان، تحتملان على التوالي
مسؤولية إنشاء تنبيه جديد واستعادة التنبيه الموجود.

5.2.3 تخزين سجلات التنبيه بشكل دائم

تحتفظ سجلات التنبيه السياق الكامل في لحظة التشغيل، بما في ذلك نوع
الحساس وحدود الحدود وقائمة أخذ العينات الفعلية، مما يوفر الأساس الأصلي
للمراجعة والتدقيق لاحقاً. يصف نموذج البيانات الخاص به الجدول 8.

جدول 8: هيكل جدول `alarm_log`

الوصف	القيود	نوع البيانات	اسم الحقل
مفتاح أساسي متزايد تلقائي	PRIMARY AUTOINCREMENT	INTEGER	id
وقت تشغيل التنبيه	NULL NOT	DATETIME	timestamp
نوع الحساس	NULL NOT	TEXT	sensor_type
نوع التنبيه: HIGH/LOW	NULL NOT	TEXT	alarm_type
الحد المشغل	NULL NOT	REAL	threshold
القيمة الفعلية عند التشغيل	NULL NOT	REAL	actual_value
الحالة: ACTIVE/RESOLVED	'ACTIVE' DEFAULT	TEXT	status
وقت الاستعادة		DATETIME	resolved_at

5.3 نظام تصور الويب

يشكل طبقة تصور الويب جسراً بين المستخدم ومحطة المراقبة الافتراضية:
بعد عرض بيانات الحساسات السفلية عبر الرسوم البيانية ولوحات القياس،
يمكن للمستخدمين أيضاً تعديل معلومات البيئية عبر لوحة التحكم وعرض إشعارات
الحالات الشاذة في لوحة التنبيه.

5.3.1 تصميم مسارات Flask

تحتمل Flask مسؤولية خدمة الويب الخلفية، وتكشف عن مجموعة من واجهات
API RESTful للاستخدام من قبل الواجهة الأمامية. يوضح الجدول 9 طريقة HTTP
والمسار والوظيفة لكل نقطة نهاية.

جدول 9: تصميم مسارات API RESTful في Flask

الوصف	الوظيفة	المسار	الطريقة
إرجاع صفحة تصور الويب	الصفحة الرئيسية	/	GET
إرجاع أحدث قراءات لجميع	أحدث البيانات	/api/sensors/latest	GET
إرجاع البيانات التاريخية ضمن نطاق زمني محدد	البيانات التاريخية	/api/sensors/history	GET
إرجاع قائمة التنبيهات غير النشطة	التنبيهات النشطة	/api/alarms/active	GET
إرجاع جميع سجلات التنبيه (ي دعم التوسيم على صفحات)	سجل التنبيهات	/api/alarms/history	GET
استقبال أوامر إعدادات	التحكم البيئي	/api/control	POST
إرجاع معلومات إحصائية لفترة محددة	البيانات الإحصائية	/api/stats	GET

5.3.2 تنسيق استجابة RESTful JSON

يحتوي كل جسم JSON مُرَجَّع من واجهة برمجة التطبيقات على ثلاث مفاتيح علوية ثابتة: status و data و message. تأخذ status قيمة 'ok' أو 'error' مما يعكس نتيجة معالجة الطلب؛ تحمل data الحمولة الفعلية على مستوى الأعمال؛ يتم ملء message فقط في السيناريوهات الشاذة بوصف خطأ قابل للقراءة. مثال على استجابة النجاح:

```

{
  "ko": "sutats",
  "atad": {
    "erutarepmet": 5.62,
    "ytidimuh": 2.85,
    "erusserp": 8.3101,
    "ecnanimulli": 0.023,
    "ytilauq_ria": 5.021,
    "level_esion": 3.54,
    "pmatsemit": "00:03:90T51-10-4202"
  },
  "egassem": "llun"
}
```

Listing 10: مثال على استجابة نجاح API

5.3.3 تكوين مخطط الخط المزدوج في Chart.js

تختار الواجهة الأمامية Chart.js لعرض منحنى الاتجاه في الوقت الفعلي. لعرض اتجاهات تغير حساسين مترابطين على نفس اللوحة، يستخدم النظام مخطط خط مزدوج ذو محور Y مزدوج (Dual Line Chart). Line Chart، يتم تقديم شيفرة تهيئة Chart.js لاتجاه الحرارة والرطوبة المشترك:

```

1 | ;)'d2'(txetnCteg.)'trahCmuHpmet'(dIyBtneleEteg.tnemucod = xtc tsnoc
2 | { ,xtc(trahC wen = trahCmuHpmet tsnoc
3 |     , 'enil' : epyt
4 |     { : atad
5 |     , [ : slebal
6 |     [ : stesatad
7 |     {
8 |     , 'درجة الحرارة' : lebal
9 |     , [ : atad
10 |     , '231 ,99 ,552(bgr' : roloCredrob
11 |     , '1.0 ,231 ,99 ,552(abgr' : roloCduorgkcab
12 |     , 'pmet-y' : DIsixAy
13 |     3.0 : noisnet
14 |     , }
15 |     {
16 |     , 'الرطوبة' : lebal
17 |     , [ : atad
18 |     , '532 ,261 ,45(bgr' : roloCredrob
19 |     , '1.0 ,532 ,261 ,45(abgr' : roloCduorgkcab
20 |     , 'ytidimuh-y' : DIsixAy
21 |     3.0 : noisnet
22 |     }
23 |     ]
24 |     , }
25 |     { : snoitpo
26 |     , eurt : evisnopser
27 |     , } eslaf : tcesretni , 'xedni' : edom { : noitcaretni
28 |     { : selacs
29 |     { : 'pmet-y'
30 |     , 'raenil' : epyt
31 |     , eurt : yalpsid
32 |     , 'tfel' : noitisop
33 |     } ' )C( 'درجة الحرارة' : txet , eurt : yalpsid { : eltit
34 |     , }
35 |     { : 'ytidimuh-y'
36 |     , 'raenil' : epyt

```

```

,eur:yalpsid | 37
,'thgir':noitisop | 38
} ' )HR % ( 'الرطوبة':txet ,eur:yalpsid { :eltit | 39
} | 40
} | 41
} | 42
; ) } | 43

```

Listing 11: تكوين مخطط الخط المزدوج في Chart.js

يستخدم المخطط الخط المزدوج yAxisID لتعريف منحنى درجة الحرارة إلى المحور Y الرئيسي ومنحنى الرطوبة إلى المحور Y الثانوي، وبالتالي إلغاء مشكلة الضغط البصري الناتجة عن اختلاف الوحدات. يتم تعيين tension على 0.3 مما يحافظ على خصائص نقاط الانعطاف في البيانات مع جعل الخطوط أكثر سلاسة بشكل عام.

5.3.4 آلية الاستطلاع كل ثانية

تقوم الواجهة الأمامية ببدء استطلاع للواجهة الخلفية بفترة 1000 مللي ثانية (ثانية واحدة) عبر setInterval، وتسحب بنشاط أحدث قراءات الحساسات وحالة التنبيه، وتحقق التحديث شبه الفوري للواجهة.

```

// كل 0001 مللي ثانية ( واحدة ) استطلاع مرة واحدة | 1
{ >= } ((lavretnItes | 2
) 'tsetal/srosnes/ipa/' (hctef | 3
)) (nosj.r >= r(neht. | 4
{ >= atad(neht. | 5
{ ) 'ko' === sutats.atad( fi | 6
;) atad.atad(sdraCrosneSetadpu | 7
;) atad.atad(strahCetadpu | 8
} | 9
}) | 10
;)) rre , ':rorre gnilloP' (rorre.elosnoc >= rre(hctac. | 11
| 12
) 'evitca/smrala/ipa/' (hctef | 13
)) (nosj.r >= r(neht. | 14
{ >= atad(neht. | 15
{ ) 'ko' === sutats.atad( fi | 16
;) atad.atad(lenaPmralAetadpu | 17
} | 18
; ) } | 19
; ) 0001 , } | 20

```

Listing 12: تنفيذ آلية الاستطلاع في الواجهة الأمامية

تقوم الواجهة الأمامية باستطلاع نقاطي النهاية `/api/sensors/latest` و `/api/alarms/active` بشكل متوازي، مما يضمن محادثة قيم البطاقات ومنحنيات الاتجاه وقائمة التنبيه مع حالة الواجهة الخلفية. بالمقارنة مع دفع WebSocket، يتنازل الاستطلاع قليلاً في الفورية، لكنه خفيف الوزن في التنفيذ وواسع التوافق مع المتصفحات، وهو مؤهل تماماً في سيناريوهات التحديث على مستوى الثانية. يمكن الترحيل إلى SSE أو WebSocket في التكرارات اللاحقة لتقليل النفقات التشغيلية بشكل أكبر.

5.4 تصميم لوحة التحكم

يقوم المستخدمون بإعادة كتابة معلومات البيئية الافتراضية مباشرة في المتصفح عبر لوحة التحكم، ومراقبة تغيرات إخراج الحساس في ظروف تشغيل محددة في الوقت الفعلي. تعد هذه القدرة بالغة الأهمية بشكل خاص للعرض الصافي واستكشاف الأخطاء.

5.4.1 تصميم آلية IPC

تقوم دورة جمع بيانات MCU الافتراضي في خيط Python مستقل، بينما تعمل خدمة الويب Flask في الخيط الرئيسي، ويجب على الاثنين الاعتناء على قناة عبر الخيوط لتبادل أوامر التحكم. تختار النظام مخطط IPC قائم على مشاركة الملفات: يعمل ملف `env_control.txt` كوسيط لتبادل البيانات ثنائي الاتجاه. تم اعتماد IPC المبني على الملفات بناءً على أربعة اعتبارات: أولاً، لا حاجة لإدخال تبعيات إضافية مثل Redis أو ZeroMQ، مما يقلل من تعقيد النظام؛ ثانياً، يتم الاحتفاظ بمحتوى الملف بعد إعادة تشغيل العملية، ويتمتع بقدرة تخزين دائم للحالة بشكل طبيعي؛ ثالثاً، يكون تنسيق النص مريحاً للمطورين للعرض المباشر والتعديل اليدوي، وتكون تكلفة التصحيح منخفضة؛ رابعاً، تعد قراءة وكتابة الملفات قدرة أساسية في غالبية لغات البرمجة، وتكون مقاومة الترحيل أقل عند إدخال وحدات متغيرة في المستقبل.

يستخدم ملف `env_control.txt` تنسيق أزواج المفاتيح والقيم البسيطة، مع معامل واحد في كل سطر، بالتنسيق: `key=value`

```
0.52=erutarepmet | 1
0.06=ytidimuh | 2
52.3101=erusserp | 3
0.005=ecnanimulli | 4
0.001=ytilauq_ria | 5
0.04=level_esion | 6
0.5=deeps_dniw | 7
```

Listing 13: مثال على تنسيق ملف التحكم البيئي

يحتفظ env_control.txt على سبعة معلومات بيئية. يقوم خيط الجمع بقراءة هذا الملف في بداية كل دورة أخذ عينات، وتحديث إدخال البيئية الحساسات الافتراضية إلى أحدث القيم؛ تقوم الواجهة الخلوية للوي ب إعادة كتابة القيمة المقابلة بعد استلام طلب POST، وتصبح المعلومات الجديدة فعالة في دورة الجمع التالية.

5.4.2 تحليل أمر set

تستقبل نقطة النهاية /api/control طلب POST بتنسيق JSON، وتستخرج معلومات البيئية المراد تعديله، ثم تكتبها مرة أخرى في ملف env_control.txt. يتبع منطق التحليل دلالة التحديث الجزئي: يتم تحديث عدد المفاتيح التي تظهر في جسم الطلب فقط، وتحافظ المفاتيح الأخرى على حالتها الأصلية، لتجنب مطالبة المستخدم بإرسال معلومات كاملة في كل مرة.

```

1  )] 'TSOP' [=sdohtem , 'lortnoc/ipa/' (etuor.ppa@
2      :)(tnemnorivne_tes fed
3      "" #"" تعيىنمعلماتالبيئية
4      )(nosj_teg.tseuqer = atad
5      :atad ton fi
6      ,))'atad oN' : 'egassem' , 'rorre' : 'sutats' {(yfinosj nruter
7      004
8      # قراءة معلمات البيئية الحالية
9      )(elif_vne_daer = vne
10
11     # تحديث المعاملات المحددة
12     , 'erusserp' , 'ytidimuh' , 'erutarepmet' [ = syek_dilav
13     , 'level_esion' , 'ytilauq_ria' , 'ecnanimulli'
14     ] 'deeps_dniw'
15     :syek_dilav ni yek rof
16     :atad ni yek fi
17     )]yek[atad(taolf = ]yek[vne
18
19     # كتابة الملف
20     )vne(elif_vne_etirw
21
22     ))vne : 'atad' , 'ko' : 'sutats' {(yfinosj nruter

```

Listing 14: واجهة إعداد معلومات البيئية

يتكون مسار تنفيذ `set_environment` من ثلاث خطوات: تحميل ملف البيئية الموجود أولاً، ثم الكتابة فوق الإدخالات المقابلة حسب أزواج المفاتيح والقيم في الطلب، وأخيراً كتابة النتيجة مرة أخرى إلى القرص. تعمل القوائم البيئية `valid_keys` مع التحويل الإجباري `float` على حجب المفاتيح غير القانونية والمدخلات ذات الأنواع الخاطئة.

5.4.3 قيود نطاق المعاملات

تفتقر محاكاة الحساسات إلى القدرة على التسامح الفيزيائي مع الإدخالات المتطرفة، وبالتالي يحدد الواجهة الخلفية نطاقات صالحة لكل من المعلمات البيئية السبعة. عندما يُرسل المستخدم قيمة تتجاوز الحدود، تُرجع واجهة برمجة التطبيقات رموز الحالة 400 مع رسالة خطأ، ويطلب التصحيح وإعادة المحاولة. [يوضح الجدول 10](#) الحدود الدنيا والعليا والقيم الافتراضية المحددة لكل معاملة.

جدول 10: النطاقات الصالحة لمعاملات البيئية

الوحدة	القيمة الافتراضية	الحد الأعلى	الحد الأدنى	اسم المعامل
C°	25.0	+85.0	-40.0	temperature
RH %	50.0	100.0	0.0	humidity
hPa	1013.25	1100.0	300.0	pressure
lux	500.0	65535.0	0.0	illuminance
ppm	100.0	10000.0	0.0	air_quality
dB	40.0	150.0	0.0	noise_level
m/s	5.0	100.0	0.0	wind_speed

يصمم نطاق كل معاملة بدقة وفقاً لأوراق بيانات الحساسات الحقيقية والقوانين الفيزيائية: تتوافق حدود درجة الحرارة العليا والدنيا مع نطاق التشغيل المُعلن لـ BMP280؛ يتم تقييد الرطوبة بين 0-100% لأن تجاوز هذا النطاق لا معنى له فيزيائياً؛ يمتد نطاق الضغط الجوي عبر السيناريوهات المتطرفة من الضغط المنخفض في المرتفعات إلى الضغط المرتفع عند مستوى سطح البحر؛ يتوافق الحد الأعلى lux 65535 للإضاءة تماماً مع النطاق الكامل لوضع H-Res في BH1750. تحجب هذه الحدود الغالبية العظمى من الإدخالات الخاطئة، وتحافظ على معقولية إخراج المحاكاة.

6 اختبار النظام والتحقق

للتحقق مما إذا كانت وحدات منصة الاعداد الافتراضي STM32 وبرامج تشغيل الحساسات وتخزين البيانات ونظام التنبيه وتصوير الويب قد وصلت إلى المؤشرات المتوقعة، يجب بناء نظام اختبار منهجي وتنفيذ التحقق متعدد

الأبعاد. فيما يلي، يتم الشرح من خمسة جوانب: بيئة الاختبار والتغطية الوظيفية ومعايير الأداء ودقة الحساس وموثوقية التنبيه.

6.1 بيئة الاختبار

يوضح الجدول 11 تكوين بيئة اختبار النظام.

جدول 11: تكوين بيئة اختبار النظام

البنية	التكوين
التشغيل النظام	Windows 11 Professional 64-bit
المعالج	Intel Core i7-12700H @ 2.3 GHz
الذاكرة	16 GB DDR5
إصدار Python	Python 3.11.6
إصدار Flask	Flask 3.0.0
إصدار SQLite	SQLite 3.42.0
المتصفح	Google Chrome 120.0

جهاز الاختبار التجريبي هو حاسوب محمول مزود بـ i7-12700H Core Intel وذاكرة DDR5، GB 16 وهو تكوين متوازن بشكل أساسي مع بيئة التطوير السائدة حالياً. يتم تنفيذ جميع عمليات الاختبار محلياً في حلقة مغلقة، مع استبعاد تدخل الأخير الشبكة الواسعة والتقلبات في حمل الخادم البعيد.

6.2 الاختبار الوظيفي

يركز الاختبار الوظيفي على التحقق من سلوك النظام حسب مواصفات المتطلبات. يغطي تصميم حالات الاستخدام تهيئة MCU الافتراضي وقراءة بيانات الحساسات وتحميل صفحة الويب وسحب بيانات API وتشغيل حدود التنبيه وتشغيل لوحة التحكم. يوضح الجدول 12 ملخص نتائج الاختبار.

جدول 12: نتائج الاختبار الوظيفي

الرقم	الاختبار بند	خطوات الاختبار	النتيجة
FT-01	MCU بدء	تهيئة VirtualSTM32 مثيل	النظام بدء
FT-02	ADC تمكين	تكوين ساعة ADC وبت التمكين	ADC دخول
FT-03	DHT11 قراءة	استدعاء دالة dht11.read() طريق	قيمة إرجاع
FT-04	MQ-135 قراءة	الانتظار حتى التسخين ثم استدعاء read()	قيمة إرجاع
FT-05	BH1750 قراءة	إرسال أمر القياس ثم القراءة	قيمة إرجاع
FT-06	BMP280 قراءة	قراءة معرف الشريحة ثم الحصول على البيانات	ضغط إرجاع
FT-07	الضوضاء قراءة	استدعاء دالة noise.read() طريق	رجع
FT-08	للويب الرئيسية الصفحة إلى الوصول	إلى الوصول http://localhost:5000/ عبر المتصفح	صفحة تحميل
FT-09	API بيانات على الحصول	GET طلب /api/sensors/latest	ت إرجاع
FT-10	العلوية الحرارة تنبيه تشغيل	تعيين temperature=40	إنشاء
FT-11	المنخفض الضغط تنبيه تشغيل	تعيين pressure=920	إنشاء
FT-12	التحكم لوحة تشغيل	تعديل معلمات البيئية عبر واجهة الويب	تحديث

جميع حالات الاستخدام الـ 12 في الجدول 12 اجتازت القبول. تم تأكيد طبقة برامج تشغيل الحساسات (FT-03) إلى (FT-07) بأن الحساسات الافتراضية الخمسة يمكنها إرجاع الكميات الفيزيائية بشكل صحيح؛ تم تأكيد طبقة خدمة الويب (FT-08، FT-09) بأن عرض الصفحة وواجهة JSON تستجيب بشكل طبيعي؛ تم تأكيد طبقة منطق التنبيه (FT-10، FT-11) بأن تجاوز الحدود العليا والدنيا يمكن اكتشافه؛ تم تأكيد طبقة التحكم (FT-12) بأنه يمكن إعادة كتابة ملف معلمات البيئية عن بُعد.

6.3 اختبار الأداء

يركز اختبار الأداء على زمن الاستجابة واستهلاك الموارد، والتحقق مما إذا كان النظام لا يزال قادراً على تلبيبة متطلبات الفورية على مستوى الثانية في سينياريوهات التشغيل المستمر.

6.3.1 زمن استجابة API

باستخدام مكتبة requests في Python، يتم إرسال 100 طلب متواصل إلى نقاط النهاية الخلفية لـ Flask، مع تسجيل زمن الذهاب والإياب في كل مرة وحساب القيم الإحصائية. يوضح الجدول 13 نتائج القياس.

جدول 13: نتائج اختبار زمن استجابة API

API نهاية نقطة	الاستجابة زمن متوسط	الأدنى الحد	الأقصى الحد
GET /api/sensors/latest	12.5 ms	8.2 ms	28.4 ms
GET /api/sensors/history	45.3 ms	32.1 ms	112.6 ms
GET /api/alarms/active	9.8 ms	6.5 ms	22.1 ms
POST /api/control	15.2 ms	10.3 ms	35.7 ms
GET /api/stats	38.6 ms	25.4 ms	98.3 ms

يتم التحكم في متوسط زمن استجابة جميع نقاط النهاية ضمن 100 مللي ثانية، مما يتوافق مع خط الأساس للأداء المحدد مسبقاً. تستعلم /api/sensors/latest و /api/alarms/active عن أحدث سجل واحد فقط، ويستغرق المتوسط حوالي 10 مللي ثانية؛ يحتاج /api/sensors/history و /api/stats إلى فحص النطاق الزمني وتنفيذ عمليات التجميع، ويرتفع متوسط الوقت المستغرق إلى 40-45 مللي ثانية، وتأتي الزيادة في النفقات بشكل أساسي من الاسترجاع في النطاق الزمني للقاعدة البيانات والحوسبة المجمعة.

6.3.2 استقرار دورة الجمع

سيؤدي اهتزاز دورة الجمع بشكل مباشر إلى انعكاسه على سلسلة منحنى الاتجاه، وسيؤثر أيضاً على دقة وقت حكم التنبيه. سجل التجربة بشكل مستمر الوقت الفعلي المستغرق في تنفيذ 100 دورة، واستخدم الدورة المستهدفة $T_{sample} = 2.0$ ثانية كمعيار لتقويم حجم الانحراف.

كان متوسط الدورة لـ 100 مرة أخذ عينات 2.003 ثانية، والانحراف المعياري 0.018 ثانية، والانحراف الأقصى الإيجابي 0.052 ثانية. يكمن سبب الاهتزاز في التقلبات في الوقت المستغرق في كتابة SQLite مرة واحدة ومنطق حكم التنبيه؛ حتى عند تراكم البيانات التاريخية إلى حجم معين، لا يزال وضع WAL قادراً على الحفاظ على مستوى منخفض من وقت انتظار القفل. من حيث الأداء العام، يكون استقرار الدورة كافياً لدعم المراقبة في الوقت الفعلي، ويمكن لمنحنى الاتجاه إعادة إنتاج اتجاهات معلومات البيئية بأمانة.

6.3.3 استهلاك الذاكرة

بمساعدة tracemalloc المدمج في Python ومكتبة psutil الخارجية، يتم تتبع
بصمة ذاكرة النظام بشكل مستمر. بعد التشغيل المستمر لمدة ساعة واحدة،
يوضح الجدول 14 تفاصيل استهلاك كل وحدة.

جدول 14: نتائج اختبار استهلاك الذاكرة

الوحدة	الذاكرة استهلاك
Flask الويب خدمة	ميجابايت 45 حوالى
والحساسات الافتراضية MCU مثيل	ميجابايت 12 حوالى
البيانات جمع خيط	ميجابايت 8 حوالى
SQLite بيانات لقاعدة البيانات التخزين ذاكرة	ميجابايت 5 حوالى
الأمامية الواجهة صفحة موارد	المتصفح (جانب ميجابايت 3 حوالى
المجموع	ميجابايت 73 حوالى

يبلغ إجمالي استهلاك ذاكرة النظام حوالي 73 ميجابايت، منها حوالي 45 ميجابايت
تخص خدمة الويب Flask، وينتج هذا الجزء بشكل أساسي عن وقت تشغيل مفسر
Python وهيكل إطار عمل Flask نفسه. يبلغ 73 ميجابايت حمولة خفيفة للغاية
في بيئات الحوسبة الحديثة، ويمكن نشره بسلسلة حتى على أجهزة الحافة
مثل Pi. Raspberry

6.4 التحقق من دقة الحساس

يتمد مصداقية محطة المراقبة الافتراضية إلى حد كبير على دقة محاكاة
الحساس. لقياس هذه المؤشرات بشكل كمّي، يتم إجراء مقارنة عرضية بين أداء
خطأ الحساسات الافتراضية والدقة المُعلنَة في أوراق البيانات الرسمية.

6.4.1 تحليل مقارنة الدقة

يوضح الجدول 15 ملخص فروق الدقة بين الحساسات الافتراضية والشرائح
الحقيقية.

جدول 15: مقارنة الدقة بين الحساسات الافتراضية والحساسات الحقيقية

المحاكاة الدقة	الحقيقي الدقة	المقاسة المعلمة	الحساس	ملاحظات
$\pm 0.8^{\circ}\text{C}$	$\pm 2^{\circ}\text{C}$	الحرارة درجة	DHT11	من أفضل
$\pm 1.5\% \text{ RH}$	$\pm 5\% \text{ RH}$	الرطوبة	DHT11	من أفضل
$\pm 8\%$	$\pm 15\%$	الغاز تركيز	MQ-135	من أفضل
$\pm 5\%$	$\pm 20\%$	الإضاءة	BH1750	من أفضل
$\pm 0.5 \text{ hPa}$	$\pm 1 \text{ hPa}$	الجوي الضغط	BMP280	من أفضل
$\pm 0.3^{\circ}\text{C}$	$\pm 0.5^{\circ}\text{C}$	الحرارة درجة	BMP280	من أفضل
$\pm 1.5 \text{ dB}$	$\pm 3 \text{ dB}$	الصوتي الضغط مستوى	الضوضاء حساس	من أفضل

كما يمكن رؤيته من الجدول 15، فإن نطاقات خطأ جميع الحساسات الافتراضية أضيّق من الدقة المُعلّنة للشرائح الحقيقية. تستبعد المحاكاة البرمجية مصادر التداخل الفيزيائية مثل الانحراف الحراري وشيخوخة المكونات والتقارب الكهرومغناطيسي وتموج مصدر الطاقة، ويتكون ضوضاء الإدراج فقط من المولد العشوائي الغاوسي، وبالتالي تكون الاتساقية والقابلية للتكرار أفضل بكثير من العتاد الحقيقي. لا يُضعف دقة المحاكاة الأعلى من الدقة الحقيقية قيمة المنصة في التدريب والتحقق. في بيئة الصرف، يسهل على الطلاب التعرف على قواعد الارتباط بين استجابة الحساس والمتغيرات البيئية عندما تكون الضوضاء أقل، وتجنب إخفاء الخصائص الرئيسية بواسطة الأخطاء العشوائية للعتاد الحقيقي. إذا كان من الضروري الاقتراب أكثر من ظروف التشغيل الحقيقية، يكفي زيادة الانحراف المعياري σ للضوضاء الغاوسية لإعادة الحساس الافتراضي إلى مستوى خطأ مشابه للشريحة الحقيقية، وإجراء اختبارات أكثر عمليّة.

6.4.2 اختبار الاستقرار على المدى الطويل

تم اختياريًا BMP280 الافتراضي لإجراء اختبار استقرار مستمر لمدة 24 ساعة: تم قفل معلمات البيئية عند $P = 1013.25 \text{ hPa}$ و $C, T = 25.0^{\circ}$ وكانت دورة أخذ العينات 2 ثانية. أظهرت الإحصائيات لـ 43200 نقطة بيانات أن متوسط الضغط الجوي كان 1013.28 hPa والانحراف المعياري 0.42 hPa والانحراف الأقصى الإيجابي $+1.8 \text{ hPa}$ ، والانحراف الأقصى السلبي -1.5 hPa . تتوافق توزيعات نقاط أخذ العينات مع القانون الطبيعي، ولم يظهرا انحرافاً أحادي أو انحرافاً منهجي، مما يثبت الاستقرار العددي للحساس الافتراضي أثناء التشغيل طويل المدى.

6.5 التحقق من وظيفة التنبؤ

تحدد موثوقية وحدة التنبؤ ما إذا كان المستخدمون قادرون على الحصول على إشعارات والتصرف في الوقت المناسب عند تدهور البيئية. فيمالي، يتم تصميم سيناريوهين نموذجيين للتحقق من السلسلة الكاملة لتشغيل التنبؤ وعرضه واستعادته.

6.5.1 سيناريو تنبيه الحرارة العالية

إعداد السيناريو: رفع درجة الحرارة البيئية إلى 40°C عبر لوحة التحكم، وهذه القيمة تتجاوز حد التنبيه الممرتفع 35°C لدرجة الحرارة. خطوات الاختبار:

1. تعيّن معامل temperature على 40.0 عبر لوحة التحكم.
2. انتظر تنفيذ دورة جمع بيانات واحدة على الأقل (حوالي 2 ثانية).
3. الاستعلام عن جدول alarm_log والتحقق مما إذا تم إنشاء سجل تنبيه درجة حرارة جدي.
4. الوصول إلى لوحة التنبيه في واجهة الويب، والتأكد من عرض معلومات التنبيه بشكل صحيح.
5. استعادة temperature إلى 25.0، والانتظار حتى استعادة التنبيه.

سجل الملاحظات: بعد تعيّن temperature على 40.0 نجح النظام في التعرف على تجاوز الحدود في الدورة الثانية لأخذ العينات، وأضاف جدول alarm_log إدخالاً جدياً بحالة status='ACTIVE' ونوع alarm_type='HIGH' وسجل actual_value 40.1°C (تم تراكب ضوضاء المحاكاة)، وكان 35.0°C عرّضت لوحة التنبيه في الويب هذه الحالة الشاذة ببطاقة حمراء بارزة. بعد استعادة درجة الحرارة إلى النطاق الطبيعى بعد حوالي 2 ثانية، تم تحويل نفس السجل تلقائياً إلى حالة RESOLVED، وتم كتابة الطابع الزمني للحظة الإلغاء في حقل resolved_at.

6.5.2 سيناريو تنبيه الضغط المنخفض

إعداد السيناريو: خفض الضغط الجوي البيئي إلى 920 hPa عبر لوحة التحكم، وهذه القيمة تقل عن حد التنبيه المنخفض 980 hPa للضغط الجوي. خطوات الاختبار:

1. تعيّن معامل pressure على 920.0 عبر لوحة التحكم.
2. انتظر تنفيذ دورة جمع بيانات واحدة على الأقل.
3. الاستعلام عن جدول alarm_log والتحقق مما إذا تم إنشاء سجل تنبيه ضغط جدي.
4. الوصول إلى لوحة التنبيه في واجهة الويب، والتأكد من عرض معلومات التنبيه بشكل صحيح.
5. استعادة pressure إلى 1013.25، والانتظار حتى استعادة التنبيه.

سجل الملاحظات: بعد تعيّن pressure على 920.0 نجح النظام في التعرف على تجاوز الحد الأدنى، وتم إنشاء إدخال تنبيه من نوع alarm_type='LOW' وكانت

hPa. 919.6 actual_value يستخدم الطرف الواجهة الأمامية علامة بترتقالية لتميز التنبيه المنخفض (يستخدم التنبيه المرتفع اللون الأحمر). بعد استعادة الضغط الجوي، تم وضع علامة على هذا التنبيه تلقائياً على أنه RESOLVED في دورات الجمع اللاحقة.

6.5.3 سيناريوهات متزامنة متعددة

التحقق من السيناريوهات المتزامنة: رفع temperature إلى 42.0°C و air_quality إلى 800 ppm في الوقت نفسه، مما يفرض إنشاء تنبيه HIGH في بعدي درجة الحرارة وجودة الهواء في الوقت نفسه. يقوم النظام بإنشاء سجلات ACTIVE مستقلة لكل الحساسين، ولا يتداخل الطبقة قاعدة البيانات؛ تعرض لوحة التنبيه في الواجهة الرئيسية الشاذتين بشكل متوازٍ مع مطابقة الألوان والحساسات؛ عند الاستعادة لاحقاً، تكون عمليات انتقال الحالة معزولة أيضاً عن بعضها البعض. يثبت هذا السيناريو أن وحدة التنبيه لا تزال تحافظ على صحة المنطق والاستقرار التشغيلي في ظل ظروف التزامن متعددة المصادر.

6.6 عرض واجهة الواجهة

يحدد بناء المكونات والخطوط البصري لواجهة الواجهة بشكل مباشر كفاءة المستخدمين في الحصول على معلومات حالة البيئية. يقسم الطرف الأمامي للنظام إلى ستة مناطق وظيفية:

(1) **بطاقات بيانات الحساسات:** يتم توزيع 6 بطاقات بيانات أفقية في الشاشة الأولى، حيث يُعَلَّم الجزء العلوي من كل بطاقة اسم الحساس، ويعرض الجزء الأوسط القيمة الحالية والوحدة بخط كبير، وترافق الجزء السفلي سهم اتجاه (↑ صاعد، ↓ هابط، → مستقر). تكون خلفية البطاقة باللون الأخضر عندما تكون القراءة في النطاق الآمن، وتتحول تدريجياً إلى اللون الأصفر عند الاقتراب من الحدود، مما يشكل تحذيراً بصرياً بديهياً.

(2) **مخطط اتجاه الحرارة والرطوبة:** يتم وضع مخطط خط مزدوج ذو محور Y مزدوج في المنطقة اليسرى الوسطى، حيث يتتبع التسلسل الأحمر اتجاه درجة الحرارة ويتتبع التسلسل الأزرق اتجاه الرطوبة، مع المحور الأفقي كوقت أخذ العينات. عند تمرير الماوس فوق نقطة البيانات، يظهر تلميح Tooltip يعرض القيمة الدقيقة، وترافق تحذيرات المنحنيات رسوماً متحركة انتقالية سلسة.

(3) **مخطط جودة الهواء والإضاءة:** يتم وضع مجموعة أخرى من المخططات الخطية المزدوجة ذات المحور Y المزدوج بشكل متماثل في المنطقة اليمنى الوسطى، حيث يعكس المنحنى البنفسجي جودة الهواء ويعكس المنحنى الأصفر شدة الإضاءة. نظراً للفجوة الكبيرة في المستويات بين ppm و lux، تمنع المقاييس المستقلة اليسرى واليمنى ضغط التسلسلات ذات المستويات المنخفضة إلى خط مستقيم.

(4) **لوحة المعلومات الإحصائية:** تلخص الجزء السفلي اليسرى المؤشرات التجميعية لثلاث نطاقات زمنية: ساعة واحدة و 24 ساعة و 7 أيام، وتغطي أبعاد مثل المتوسط

والقيم القسوى. يتم سحب البيانات ديناميكيًا عبر /api/stats/ ويقوم المستخدمون بالتبديل بين نوافذ الإحصائيات عبر قائمة منسدلة.
(5) **لوحة التنبيه:** تنقسم الجزء السفلي إلى قسمين علوي وسفلي، حيث يُدرج القسم العلوي التنبيهات في حالة ACTIVE في الوقت الفعلي، وتجذب الخلقية الحمراء/البرتقالية الانتباه، ويُعلّم كل تنبيه النوع والحساس ولحظة التشغيل والقيمة المتجاوزة؛ يعرض القسم السفلي السجلات التاريخية بالترتيب الزمني العكسي، ويدعم تصفح الصفحات.

(6) **لوحة التحكم:** تحتوي الشريط الجانبي الأيمن على 7 مجموعات من عناصر التحكم، تتوافق مع درجة الحرارة والرطوبة والضغط الجوي والإضاءة وجودة الهواء والضوضاء وسرعة الرياح. يُعلّم بجانب كل عنصر تحكم القراءة الحالية والنطاق القانوني، ويتم اعتراض الإدخالات المتجاوزة للحدود على الفور وعرض رسالة ودية.

تستخدم الواجهة الأمامية تخطيط شبكة متجاوب، حيث تتم إعادة ترتيب البطاقات والمخططات تلقائيًا حسب عرض إطار العرض، والحفاظ على قابلية القراءة على كل من أجهزة سطح المكتب والأجهزة المحمولة. يعمد التخطيط الألوان على الرمادي الفاتح كخلفية والأزرق والأخضر كألوان رئيسية والأحمر والبرتقالي كألوان تحذير، مع تدرج طبقي، مما يساعد المستخدمين على تحديد المعلمات الرئيسية خلال 3 ثوانٍ.

7 الخلاصة والآفاق

7.1 ملخص أعمال الورقة البحثية

تناول هذه الدراسة محاكاة محطة مراقبة بيئية ذكية مبنية على STM32 كموضوع رئيسي، وقد اكتملت العمل الكامل من النظرية إلى النموذج الأولي، ويمكن تلخيص المخرجات الأساسية في النقاط الثلاث التالية:

- (1) **بناء منصة عتاد STM32 افتراضية عالية الدقة.** باستخدام STM32F103C8T6 كموضوع مرجعي، تم تنفيذ مجموعة كاملة من الملحقات الافتراضية بلغة Python، تغطي نواة Cortex-M3 وتعيين الذاكرة ذو الثلاث أقسام (المسجلات/Flash/SRAM) ومؤقت SysTick وGPIO وADC وI2C وUART وTIM. لا تعتمد هذه المنصة على أي عتاد مادي، ويمكنها تشغيل البرامج المدمجة مباشرة على أنظمة Windows وLinux وmacOS. أظهرت التجارب التحققية أن سلوك القراءة والكتابة للمسجلات الافتراضية يتماشى بدرجة عالية مع الأوصاف في دليل مرجعي STM32 الحقيقي، وهو ما يكفي لدعم التدريس في الفصول الدراسية والتحقق المبكر من النماذج الأولية.
- (2) **إنشاء إطار موحد لمحاكاة الحساسات البيئية.** تم إنجاز ترحيل برامج تشغيل لخمس أنواع شائعة من الحساسات البيئية فوق العتاد الافتراضي: DHT11 وMQ-135 وBH1750 وBMP280 وحساس الضوضاء. يوحّد الإطار على مستوى الإطار تعيين معلمات البيئية وتحويل الكميات الفيزيائية وآلية حقن الضوضاء

الغاوسية $Y = X + N(0, \sigma^2)$ ، مما يجعل الإخراج المحاكاة يتبع القوانين الفيزيائية ويحتفظ بالخصائص العشوائية الحقيقية في الوقت نفسه. تعيد برامج التشغيل إنتاج بروتوكولات الاتصال bus أحادي و I2C وتوقيت التشغيل وسلسلة معالجة الإشارة بشكل كامل، والخطأ التراكمي أفضل من الدقة المعلنه في أوراق البيانات الرسمية.

(3) **تنفيذ نظام تصورات وتحكم كامل عبر الويب.** توفر الواجهة الخلوية واجهات

API RESTful بناءً على Flask، وتتولى SQLite تخزين بيانات الحساسات وسجلات

التنبؤ وسجلات النظام بشكل دائم؛ تكمل الواجهة الأمامية باستخدام HTML/CSS/JavaScript

الأصلي مع مكتبة Chart.js عرض الخطوط الزمنية في الوقت الفعلي والتتبع

التاريخي ونوافذ التنبؤ وعناصر التحكم المنزلق لمعاملات البيئية. يتم

تحقيق الاتصال المغلق الحلقة بين لوحة التحكم و MCU الافتراضي عبر ملف

مشترك env_control.txt، حيث يمكن للمستخدمين سحب المنزلقات في الواجهة

الأمامية لمراقبة تغيرات قراءات الحساسات في دورة الجمع التالية.

أكدت مرحلة الاختبار الفعلي: أن المنصة الافتراضية يمكنها العمل باستقرار

على مدار 24/7، ويتم التحكم في انحراف دورة الجمع ضمن ± 0.052 ثانية، ومتوسط

زمن استجابة واجهة برمجة تطبيقات الويب أقل من 100 مللي ثانية، وإجمالي

نقاط ذكرة الموقع حوالي 73 ميغابايت فقط. يمكن لوحدة التنبؤ اكتشاف

تجاوز الحدود العليا والدنيا بدقة وإنشاء الأرشفة، ويتميز مستوى معالومات

الواجهة الأمامية بوضوح التدرج. يقلل هذا الحل البرمجية الصرفة تكلفة

الدخول إلى التجارب المدمجة إلى الحد الأدنى، ويوفر أيضاً صندوق رمل للتحقق

السرعي والتكرار بتكلفة صفرية للعتاد في المراحل المبكرة من مشاريع

إنترنت الأشياء.

7.2 آفاق العمل المستقبلي

على الرغم من أن النسخة الحالية من محطة المراقبة البيئية الافتراضية قد

غطت سيناريوهات التدريب الأساسية والتحقق من النماذج الأولية، إلا أنه لا

يزال هناك خمسة مسارات تستحق الاستكشاف في العمق الهندسي وتوصيل

النظام البيئي:

(1) **محاكاة نظام التشغيل FreeRTOS في الوقت الفعلي.** تدعم المنصة الحالية

فقط البرامج المكشوفة، وتعتد مدجولة المهام تماماً على الاستطلاع والمقاطعات.

يمكن ترحيل محاكاة جدول FreeRTOS في الخطوة التالية، لدعم الاستباق حسب

الأولوية والتناوب بالشرائح الزمنية ومزامنة السيميافورات واتصال قوائم الانتظار

وتخصيص الذاكرة الديناميكي، مما يمكن المتعلمين من تجربة البرمجة المتزامنة

متعددة المهام في بيئة افتراضية، وتقدير المسافة الإدراكية من الفصل

الدراسي إلى الموقع الصناعي.

(2) **دعم بروتوكول MQTT.** لا يزال الاستطلاع عبر HTTP مقبولاً في السيناريوهات

على مستوى الثانية، لكن النفاقات الاتصالية وتأخير الدفع يحدان من قابلية

التوسع في سيناريوهات إنترنت الأشياء عالية التزامن. بعد إدخال MQTT، تقوم

MCU الافتراضي بدفع بيانات الحساسات إلى Broker بصفتها ناشراً، وتعمل الواجهة الأمامية للويب والأجهزة المحمولة كمشتريين لاستقبال تحديثات الموضوع، وتحقق الدفع الحقيقي حسب الطلب. باعتباره بروتوكولاً قياسيً واقعيً في مجال إنترنت الأشياء، سيبرز إضافته بشكل مباشر قدرة المنصة على التواصل مع الأنظمة الصناعية الحقيقية.

(3) **توسيع المزيد من أنواع الحساسات.** تغطي مكتبة الحساسات الحالية خمسة أبعاد: الحرارة والرطوبة والضغط الجوي والإضاءة وجودة الهواء والضوضاء، ويمكن الاستمرار في الوصول إلى وحدات MH-Z19C CO2 ووحدات الجسيمات PMS5003 ووحدات الأشعة فوق البنفسجية GUVVA-S12SD وحساسات رطوبة التربة ومقاييس طول الأمطار ذات الميزان المائل وغيرها. بعد توسيع الأنواع، يمكن للمنصة الاندماج بسلسلة في المزيد من المجالات الرئيسية مثل البيوت الزجاجية الذكية للزراعة والمراقبة البيئية الشبكية للمدن والصحة والسلامة المهنية في المصانع.

(4) **محاكاة وضع استهلاك الطاقة المنخفض.** يدمج STM32F103C8T6 ثلاثة مستويات لاستهلاك الطاقة المنخفض: Sleep و Stop و Standby، وتتمدد أداء عمر البطارية في السيناريوهات المشغلة بالبطارية إلى حد كبير على إدارة الدقة لهذه الحالات. يمكن محاكاة تدفقات التحكم في البوابة الزمنية وإطفاء الملحقات وتعيين مصادر الإيقاظ بشكل كامل في المنصة الافتراضية في التكرارات اللاحقة، مما يمكن المتعلمين من فهم العلاقة التبادلية بين استهلاك الطاقة وتأخير الاستجابة دون استهلاك بطاريات حقيقية.

(5) **تطوير تطبيق الجوال.** على الرغم من أن واجهة الويب الحالية تملك بالفعل قدرة التوافق عبر الأنظمة الأساسية، إلا أنها لا تزال مقيدة من صندوق رمل المتصفح فيما يتعلق بالشاشات الصغرية التي تعمل باللمس والحفاظ على الخلفية والإشعارات المحلية. بعد تطوير تطبيق مصاحب بناءً على Flutter أو React Native، يمكن الوصول إلى دفع النظام والبيث الصوتي لتنبيهات الحالات الشاذة، وربط بيانات المراقبة بموقع GPS الجغرافي، وتحقيق الإدراك البيئي الحقيقي أثناء التنقل.

من منظور التدريس، تزيل وسائل المحاكاة الافتراضية القيود الصارمة لشراء المعدات وجدولة المختبرات، ويمكن للطلاب إعادة إنتاج سير عمل تطوير الأنظمة المدمجة بالكامل على أجهزة الكمبيوتر الشخصية في أي وقت؛ ومن منظور هندسي، تسمح بيئات التحقق البرمجية الصرفة بالتكرار السريع لخطط الهندسة المعمارية بتكلفة منخفضة لللغاية في المراحل المبكرة من المشروع. مع استمرار الطلب الانباض على المواهب المدمجة من صناعة إنترنت الأشياء، ستلعب منصات المحاكاة الافتراضية عالية الدقة مثل هذه دوراً متزايد الأهمية في تقصير دورة تدريب المواهب وتخفيف تكاليف التجربة والخطأ في الابتكار.

المراجع

- [1] STM32F4[M]. Sen. Yang Huoliang, Liu [1] بكي: 2019. Press, Machine China.
- [2] Atomic. دليل تطوير STM32F1: نسخة وظائف المكتبة [M]. قوانغ تشو: Atomic, 2022.
- [3] Minghui. Zhang Lihua, Wang [3] المدمجة [J]. 112-117. 2021(5): Education, Computer.
- [4] Xiaofeng. Li Zhigang, Chen [4] القوائم على إنترنت الأشياء [J]. 2020, Technologies, Microsystem and Transducer. 1-4. 39(8):
- [5] Lina. Zhang Runjing, Zhou [5] STM32[M]. ل. Proteus على القوائم على STM32[M]. بكي: 2018. Industry, Electronics of House Publishing.
- [6] Limited. ARM [6] 2021. Developer, ARM Cortex-M3[EB/OL].
- [7] STMicroelectronics. [7] 3MTS x,x201F23MTS x,x101F23MTS RM0008: دليل مرجعي 32- ARM على مقدمة MCUs xx701F23MTS و xx501F23MTS x,x301F2 2021. Community, ST bit[EB/OL].
- [8] AG. Sensirion [8] ورقة بيانات: SHT30-DIS حساس الحرارة والرطوبة [EB/OL]. Sensirion, 2020.
- [9] Bosch [9] ورقة بيانات حساس الضغط الرقمي Bosch, BMP280[EB/OL]. Sensortec. Bosch 2020.
- [10] Semiconductor. ROHM [10] BH1750FVI-E: حساس إضاءة محيط بإخراج تسلسلي رقمي 16 بت [EB/OL]. ROHM, 2019.
- [11] Electronics. Hanwei [11] ورقة بيانات حساس الغاز. Hanwei, MQ-135[EB/OL]. 2018.
- [12] [https: 2023. Projects, Pallets Start\[EB/OL\]. Quick 3.0 Flask Documentation. Flask \[12\] //flask.palletsprojects.com/](https://flask.palletsprojects.com/)
- [13] Developer Mozilla [13] 2023. Docs, Web MDN APIs[EB/OL]. Web API Fetch Network. Developer Mozilla https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- [14] [https: 2023. Chart.js, Started\[EB/OL\]. Getting 4.x Chart.js Documentation. Chart.js \[14\] //www.chartjs.org/docs/](https://www.chartjs.org/docs/)
- [15] Yang. Liu Zhihua, Wang [15] تصميم وتنفيذ نظام مراقبة بيئية لإنترنت الأشياء 45- 12(3): 2022, Technologies, Things of Internet MQTT[J]. الالقائم على بروتوكول 48.

- [16] Wei. Zhang Jianguo, Li [16] بحث في تطبيق تقنيات المحاكاة في تدريس الأنظمة
الدمجة[J]. 43(2): 2021, Education, Electronic and Electrical of Journal. 78-82.
- [17] Consortium. SQLite [17] SQLite, SQLite[EB/OL]. توثيق. 2023. <https://www.sqlite.org/docs.html>
- [18] Li. Sun Minghua, Zhao [18] تطبيق تقنية الأجهزة الافتراضية في تدريس الحساسات[J].
37(6): 2020, Management, and Technology Experimental 156-160.
- [19] Project. QEMU [19] QEMU, QEMU[EB/OL]. توثيق. 2023. <https://www.qemu.org/documentation/>
- [20] Haitao. Wang Liqiang, Yang [20] تصميم منصة تدريس تجارب مدمجة عن بُعد قائمة
على الويب[J]. 55(14): 2019, Applications, and Engineering Computer. 245-250.